SOCIALCIRCLE ANALYZER

Project Report Submitted to the

MADURAI KAMARAJ UNIVERSITY, MADURAI

In partial fulfillment of the requirement for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

SUBMITTED BY

C. JEYASHREE (Reg.No: [REGISTER NUMBER])



UNDER THE GUIDANCE OF

[GUIDE NAME, QUALIFICATIONS]

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

NPR ARTS & SCIENCE COLLEGE

NATHAM

[ACADEMIC YEAR]

NPR ARTS & SCIENCE COLLEGE

NATHAM

DEPARTMENT OF COMPUTER SCIENCE

BONAFIDE CERTIFICATE

Certified that this project report **"SOCIALCIRCLE ANALYZER"** is the bonafide work of **C. JEYASHREE ([REGISTER NUMBER])** in **Bachelor of Science in Computer Science** who carried out the project work under my supervision.

Internal Guide	Head of the Department
[GUIDE NAME]	[HOD NAME]

Submitted for the viva-voce held at NPR Arts & Science College, Natham on _____

INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I hereby declare that this project report entitled **"SOCIALCIRCLE ANALYZER"** is a record of original work done by me under the guidance of **[GUIDE NAME]**, Department of Computer Science, NPR Arts & Science College, Natham, and this project work has not formed the basis for the award of any degree or similar to candidate in any university.

C. JEYASHREE ([REGISTER NUMBER])

Place:

Date:

ACKNOWLEDGEMENT

I praise **GOD** almighty for his blessings and grace showered to me during my project work and to finish this project in a successful way. My sincere efforts have been made to accomplish the task of this project.

I express my heartfelt thanks to our beloved principal **[PRINCIPAL NAME, QUALIFICATIONS]** for giving me an opportunity to undergo this project work.

I extend my profound gratitude to **[HOD NAME, QUALIFICATIONS]**, Head of the Department, NPR Arts & Science College, Natham, for giving me an opportunity to undergo this project work and timely help.

My heartfelt thanks to my guide **[GUIDE NAME, QUALIFICATIONS]**, Assistant Professor, Department of Computer Science, NPR Arts & Science College, Natham, for her guidance and supervision throughout the period of my project work.

I express my high thankfulness to my parents and my friends for their help and encouragement throughout the successful completion of my project work.

INDEX

S.NO	CONTENTS	PAGE NO
1.	**INTRODUCTION**	1-5
	1.1 About the Project	1
	1.2 Project Description	1
	1.3 Problem Statement	2
	1.4 Objectives	2
	1.5 Scope of the Project	3
	1.6 Related Work	4
	1.7 Organisation of the Report	4
2.	**SYSTEM ANALYSIS**	6-9
	2.1 Feasibility Study	6
	2.2 Existing System	7
	2.3 Proposed System	7
	2.4 Requirement Analysis	8
3.	**SYSTEM ENVIRONMENT**	10-13
	3.1 Hardware Requirements	10
	3.2 Software Requirements	10
	3.3 Software Specification	11
4.	**SYSTEM DESIGN**	14-26
	4.1 Input Design	14
	4.2 Output Design	14
	4.3 Data Design	15
	4.4 System Architecture	17
	4.5 Data Flow Diagram	18
	4.6 Use Case Diagram	22
	4.7 Activity Diagram	23
	4.8 Sequence Diagram	24
	4.9 User Interface Design	25
5.	**SYSTEM SPECIFICATION**	27-34
	5.1 Module Description	27
	5.2 Algorithms and Techniques	29
	5.3 Worked Example of the Analysis	31
	5.4 API Specification	33
	5.5 Front-End Function Specification	33
6.	**SYSTEM IMPLEMENTATION**	35-43

S.NO	CONTENTS	PAGE NO
	6.1 Project Structure	35
	6.2 Back-End Implementation	35
	6.3 Front-End Implementation	36
	6.4 Running the Application	37
	6.5 Source Code Walkthrough	37
	6.6 Error Handling	38
	6.7 Results and Discussion	38
	6.8 Advantages of the System	42
	6.9 Limitations of the System	43
7.	**SYSTEM TESTING**	44-48
	7.1 Types of Testing	44
	7.2 Test Environment	45
	7.3 Backend Test Cases and Results	45
	7.4 Manual Test Cases for the User Interface	46
	7.5 Performance Observation	47
	7.6 Requirement Coverage	47
	7.7 Observations from Testing	48
8.	**CONCLUSION**	49
9.	**FUTURE ENHANCEMENT**	50
10.	**BIBLIOGRAPHY**	51-52
11.	**APPENDIX**	53-68

1. INTRODUCTION

1.1 ABOUT THE PROJECT

The rapid growth of online social platforms has produced large collections of user profiles, each described by its own pattern of activity. A single user may publish frequently, another may only read and like the posts of others, and a third may attract thousands of followers. Understanding how such users are similar to, or different from, one another is a common task in data analysis, and it becomes difficult when the only tool available is a long table of numbers.

SocialCircle Analyzer is a web-based data-analysis application that addresses this difficulty. The system takes a collection of user records, each described by a small set of behavioural measures, groups similar users together with the K-Means clustering algorithm, connects every user to the five users who are most similar to them, and finally displays the whole result as an interactive network graph in the web browser. Users who belong to the same group are drawn in the same colour, users with more followers are drawn larger, and clicking a user opens a panel that shows that user's individual metrics.

The application follows a client–server design. The server side is written in Python with the FastAPI framework and uses the NumPy, pandas and scikit-learn libraries for data handling and analysis. The client side is written in HTML, CSS and JavaScript and draws the graph with the open-source force-graph library. The two sides communicate through a single JSON endpoint, `/api/network`.

A note on naming. The project is reported here under the title "SocialCircle Analyzer". The prototype's browser page and sidebar carry the working name "SocialSphere" with the sub-title "K-Means Social Clustering". Both names refer to the same application, and the source code is documented exactly as it exists.

1.2 PROJECT DESCRIPTION

The goal of this project is to make the behavioural similarity between social-network users visible at a glance. The system converts raw numeric records into a coloured network so that groups of similar users, and the strength of their similarity links, can be understood without reading the underlying numbers. The implemented features of the application are listed below.

- **User data generation:** the backend creates 300 synthetic user records that belong to four behavioural personas (Techie, Sports Fan, Influencer and Lurker).
- **Feature standardisation:** six user features are rescaled with the StandardScaler so that no single feature dominates the analysis.
- **K-Means clustering:** the standardised records are divided into four clusters.
- **Similarity network:** a k-nearest-neighbour graph (five neighbours per user) provides the connections between similar users.
- **REST API:** the endpoint `GET /api/network` returns the users as nodes and the connections as links in JSON form.
- **Interactive graph:** the browser draws a force-directed graph in which colour shows the cluster and node size shows the follower count.
- **Statistics and legend:** the sidebar shows the number of users and connections and a colour legend for the clusters.
- **Node detail panel:** clicking a user shows the name, persona, followers, posts per day and likes of that user.
- **Re-Cluster control:** a button that requests the analysis from the server again and redraws the graph.

It is important to state the boundaries of the current implementation clearly. The project analyses **synthetic data generated inside the program**. It does not read uploaded CSV files, it does not analyse messages between contacts, and it does not measure changes over time. These possibilities are discussed as future enhancements in Chapter 9 and are not described as implemented features anywhere in this report.

1.3 PROBLEM STATEMENT

Behavioural data about users is usually stored as rows and columns. When the number of users grows to a few hundred, it is very hard to see from such a table which users behave alike, which users form natural groups and which users stand out. Simple sorting by one column, for example by the number of followers, hides the fact that users differ in several measures at the same time. A method is therefore needed that considers all measures together, groups the users automatically, and presents the result in a form that a non-expert can explore.

1.4 OBJECTIVES

The main objectives of the project are:

1. To design a small web application that analyses user-behaviour data and presents the result visually.
2. To prepare a multi-feature user data set and standardise it so that all features contribute fairly to the analysis.
3. To group users into meaningful clusters using the K-Means algorithm of the scikit-learn library.
4. To connect each user with the most similar users using a k-nearest-neighbour graph and to expose the result through a REST API.
5. To display the network as an interactive graph in which colour, size and click actions convey information.
6. To build the system only from freely available, open-source technologies so that it can run on an ordinary computer.

1.5 SCOPE OF THE PROJECT

The scope of the current version is deliberately limited to a working prototype. Table 1.1 separates what the application does today from what it does not do, so that the report does not claim more than the software delivers.

Table 1.1: Scope of the Current Implementation

Area	Implemented in this project	Not implemented (future work)
Data source	Synthetic users generated in `generate_mock_data()`	Upload of CSV files or connection to real social-media data
Analysis	Standardisation, K-Means (k = 4), 5-nearest-neighbour graph	Automatic selection of k, cluster-quality scores, trend analysis
Time	Single snapshot of the data	Interaction trends and changes over time
Interaction data	User-level measures: posts per day, likes given, followers, three interest flags	Message-level or contact-to-contact interaction counts
Interface	Force-graph view, statistics, legend, node panel, Re-Cluster button	Search, filtering, export of results, settings panel
Storage	No database; data is created in memory for every request	Persistent storage of users and results
Users and security	Single anonymous user; open CORS policy	Login, roles and access control

1.6 RELATED WORK

The ideas used in this project are well established in data analysis. **Social network analysis** treats people as nodes and their relations as links and studies the resulting graph structure (Wasserman and Faust, 1994; Newman, 2018). **Cluster analysis** divides objects into groups so that objects in the same group are more similar to each other than to objects in other groups; Jain (2010) reviews the development of clustering methods over fifty years and identifies K-Means as one of the most widely used. The K-Means procedure itself goes back to Lloyd (1982) and MacQueen (1967), and the k-means++ seeding method that scikit-learn uses by default was proposed by Arthur and Vassilvitskii (2007).

The **nearest-neighbour rule** was formalised by Cover and Hart (1967). In this project it is not used for classification but to create a similarity graph in which every user is linked to the five users whose standardised features are closest to their own. Finally, **force-directed drawing** places strongly connected nodes near each other and pushes unconnected nodes apart, which makes clusters of connected users visible without manual layout. In the browser this drawing is performed by the force-graph library.

SocialCircle Analyzer combines these established techniques in a compact application. It is not offered as a new algorithm; its contribution is the working integration of data preparation, clustering, graph construction, a REST interface and an interactive front end.

1.7 ORGANISATION OF THE REPORT

Table 1.2: Organisation of the Report

Chapter	Title	Content
1	Introduction	About the project, description, problem statement, objectives, scope, related work
2	System Analysis	Feasibility study, existing and proposed system, functional and non-functional requirements
3	System Environment	Hardware and software requirements; specification of the technologies used
4	System Design	Input and output design, data design, architecture, data flow, use case, activity and sequence diagrams
5	System Specification	Module description, algorithms and techniques, API specification
6	System Implementation	Implementation of back end and front end, execution steps, results, advantages and limitations

Chapter	Title	Content
7	System Testing	Testing strategy, executed backend tests, manual test cases, performance observations
8	Conclusion	Summary of the work
9	Future Enhancement	Possible extensions (not implemented)
10	Bibliography	Books, papers and documentation used
11	Appendix	Screen shots, sample coding and sample API output

2. SYSTEM ANALYSIS

2.1 FEASIBILITY STUDY

A feasibility analysis usually involves a thorough assessment of the operational, financial and technical aspects of a proposal. The feasibility study tests whether the needs of the users can be satisfied with the available software and hardware technology, whether the system is affordable and whether it will be accepted by its users. It is carried out at the earliest possible time so that a decision can be taken on whether to proceed with detailed analysis and development.

When a new project is proposed it normally goes through a feasibility assessment. The facts considered for SocialCircle Analyzer are:

- Technical feasibility
- Economic feasibility
- Behavioural (operational) feasibility

Technical Feasibility

Technical feasibility deals with the hardware and software that the system needs and with whether the chosen technology can meet the requirements. SocialCircle Analyzer depends only on the Python language and a small number of widely used libraries. The whole analysis for 300 users runs in memory and, as measured in Chapter 7, completes in a fraction of a second on an ordinary processor. The points established during the technical study are:

- The software runs on a standard desktop or laptop operating system (the project was developed with Python 3.12 in a Windows virtual environment).
- All required libraries (FastAPI, Uvicorn, pandas, NumPy, scikit-learn) are available from the Python Package Index and are installed with a single command.
- The client needs only a modern web browser with JavaScript enabled.
- The system can be expanded, for example by replacing the synthetic data generator with a CSV reader, without changing the rest of the pipeline.

Economic Feasibility

Economic feasibility compares the development and operating cost of the system with its benefits. Every technology used in the project is open source and free of licence charges. The application needs no database server, no paid cloud service and no special hardware. The only cost is the developer’s time. The benefit is a tool that presents behavioural clusters

visually, which would otherwise require manual spreadsheet work or specialised software. The project is therefore economically feasible.

Behavioural / Operational Feasibility

This analysis considers how the system will work once it is installed and whether its users will accept it. The interface has only a few controls: a Re-Cluster button, a legend, and clickable nodes. No training in statistics or programming is required to read the graph, because colour and size are explained by the legend and by the detail panel. A user who can operate a web page can operate the application, so the project is operationally feasible.

2.2 EXISTING SYSTEM

In the existing way of working, behavioural data about social-network users is normally examined as a table or spreadsheet. The analyst sorts and filters columns, calculates averages, and compares users by eye. General-purpose statistical packages can perform clustering, but they require the analyst to prepare the data, write scripts or use complex menus, and the output is often a table of labels rather than a picture of the relationships between users. The drawbacks of this approach are:

1. Users are compared on one measure at a time, so combinations of measures are missed.
2. Grouping of similar users has to be done manually and is subjective.
3. Results are shown as rows and columns, which are hard to interpret when the number of users is large.
4. The relationship between one user and the users most similar to that user is not visible.
5. Specialised tools need technical skill and separate installation and configuration.

2.3 PROPOSED SYSTEM

The proposed system removes these drawbacks by automating the whole path from user records to a visual result. When the page is opened, the browser asks the server for the analysis; the server prepares the user records, standardises the features, applies K-Means clustering, builds the nearest-neighbour graph and returns the result; and the browser draws an interactive network. The advantages of the proposed system are:

1. All six features are considered together in a single clustering step.
2. Users are grouped automatically and consistently by an established algorithm.

3. The result is a coloured network in which clusters and similarity links can be seen directly.
4. Every user can be inspected with one click, and the analysis can be repeated with one button.
5. The system is lightweight, uses only free software and runs in a web browser.

Benefits of the Proposed System

The system gives students, analysts and other non-specialist users a simple way to explore clustering results. Because the analysis logic is kept in a single backend file and the presentation logic in three small front-end files, the application is easy to understand, easy to maintain and easy to extend with real data in later versions.

2.4 REQUIREMENT ANALYSIS

The requirements below were derived from the behaviour of the implemented source code. Each functional requirement corresponds to a function, endpoint or interface element that exists in the project.

Functional Requirements

Table 2.1: Functional Requirements

ID	Requirement	Implemented in
FR-01	The system shall generate 300 user records belonging to four personas.	`generate_mock_data()`
FR-02	The system shall standardise six features before clustering.	`StandardScaler` in `get_network_data()`
FR-03	The system shall assign every user to one of four clusters using K-Means.	`KMeans(n_clusters=4)`
FR-04	The system shall connect every user with its five nearest neighbours.	`kneighbors_graph(n_neighbors=5)`
FR-05	The system shall return nodes and links as JSON through `GET /api/network`.	`get_network_data()`
FR-06	The system shall serve the web page at the root address and the static files under `/static`.	`serve_index()`, `StaticFiles`
FR-07	The interface shall draw a force-directed graph coloured by cluster with node size based on followers.	`app.js`, force-graph
FR-08	The interface shall display the number of users and connections and a cluster legend.	`loadNetworkData()`, `createLegend()`

ID	Requirement	Implemented in
FR-09	The interface shall show a detail panel when a node is clicked and hide it on background click or close.	`showNodePanel()`, `hideNodePanel()`
FR-10	The interface shall allow the analysis to be requested again with the Re-Cluster button.	`btn-recluster` handler
FR-11	The interface shall inform the user if the data cannot be loaded.	`catch` block with `alert()`

Non-Functional Requirements

Table 2.2: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Usability	The interface shall be operable with mouse clicks only and shall use a legend to explain colours.
NFR-02	Performance	The analysis of 300 users shall complete quickly enough for interactive use (measured in Chapter 7).
NFR-03	Portability	The server shall run on any operating system that supports Python 3.12 and the listed libraries; the client shall run in a modern browser.
NFR-04	Maintainability	Analysis code and presentation code shall be kept in separate files (`main.py`, `app.js`, `style.css`, `index.html`).
NFR-05	Reproducibility	The generated data shall be reproducible through fixed random seeds (seed value 42).
NFR-06	Visual design	The interface shall use a dark theme with translucent panels and a consistent four-colour cluster palette.
NFR-07	Cost	The system shall use only free and open-source software.

3. SYSTEM ENVIRONMENT

3.1 HARDWARE REQUIREMENTS

SocialCircle Analyzer is a light-weight application. The analysis of 300 users needs very little memory or processing power, so no special hardware is required. The hardware configuration recommended for developing and running the project is:

Processor	:	Intel Core i3 or equivalent (dual-core, 1.6 GHz or above)
RAM capacity	:	4 GB (8 GB recommended)
Hard disk drive	:	At least 1 GB free space (the Python virtual environment is the largest part)
Keyboard	:	Standard
Mouse	:	Optical
Monitor	:	Colour display, 1366 × 768 resolution or higher

These values are recommended minimums for the application; they are not measurements of any particular machine.

3.2 SOFTWARE REQUIREMENTS

The software used for the development and execution of the project is:

Operating System	:	Windows 10 / 11 (any OS supporting Python 3.12)
Programming language	:	Python 3.12
Web framework	:	FastAPI
Application server	:	Uvicorn (ASGI server)
Analysis libraries	:	pandas, NumPy, scikit-learn
Front end used	:	HTML5, CSS3 and JavaScript
Graph library	:	force-graph (loaded in the browser)
Web browser	:	Google Chrome, Microsoft Edge or Mozilla Firefox

The Python packages required by the back end are listed in the file `backend/requirements.txt`: fastapi, uvicorn, scikit-learn, pandas, numpy, networkx and pydantic. Table 3.1 shows the versions found in the project's virtual environment and how each package is used by the source code.

Table 3.1: Python Packages of the Project

Package	Version installed	Role in the project
fastapi	0.141.1	Creates the web application, the `/api/network` endpoint and static file serving
uvicorn	0.54.0	Runs the application through `uvicorn.run(app, host, port)`
pandas	3.0.6	`DataFrame` holding the user records
numpy	2.5.3	Random number generation for the synthetic data
scikit-learn	1.9.1	`StandardScaler`, `KMeans`, `kneighbors_graph`
pydantic	2.13.5	Installed as a dependency of FastAPI; no model is defined in the project code
networkx	3.7	Listed in `requirements.txt`; not imported by the current code

The front end additionally loads three resources from the internet through CDN links in `index.html`: the force-graph script, the Font Awesome 6.4.0 icon stylesheet and the Outfit web font from Google Fonts. An internet connection is therefore needed when the page is opened.

3.3 SOFTWARE SPECIFICATION

Python

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. Its clear syntax, dynamic typing and very large collection of libraries have made it the most common language for data analysis and machine learning. In this project Python is used for every part of the server: creating the data, standardising features, clustering, building the graph and exposing the result through a web API.

FastAPI

FastAPI is a modern Python web framework for building APIs. It is built on the Starlette toolkit and the Pydantic data-validation library and follows the ASGI standard, which allows it to serve requests efficiently. A function is turned into an endpoint by placing a decorator such as `@app.get("/api/network")` above it. FastAPI also generates interactive API documentation automatically under the `/docs` address. SocialCircle Analyzer uses FastAPI for three things: the JSON endpoint, the root route that returns `index.html`, and the mounting of the `static` folder.

- Simple decorator-based routing of HTTP requests.
- Automatic conversion of Python dictionaries into JSON responses.
- Built-in `StaticFiles` and `FileResponse` classes for serving the front end.
- `CORSMiddleware` for controlling cross-origin access (configured here to allow all origins).

Uvicorn

Uvicorn is a lightning-fast ASGI web server for Python. FastAPI itself only describes the application; a server such as Uvicorn is needed to listen on a network port and pass requests to it. In `main.py` the block `if __name__ == "__main__"` calls `uvicorn.run(app, host="0.0.0.0", port=8080)`, so that the application can be started with the command `python main.py` and reached at port 8080.

pandas

pandas is an open-source library for data manipulation and analysis, originally developed by Wes McKinney (2010). Its main structure, the `DataFrame`, stores tabular data in labelled rows and columns. The project uses a `DataFrame` to hold the 300 generated user records, to select the six clustering features (`df[features]`), to add the `cluster` column, and to iterate over the rows when building the JSON response.

NumPy

NumPy is the fundamental package for numerical computing in Python and supplies the N-dimensional array on which pandas and scikit-learn are built (Harris et al., 2020). The project uses NumPy's random-number generator: `np.random.seed(42)` makes the data reproducible and `np.random.normal(mean, deviation)` draws values for posts, likes and followers from a normal (bell-shaped) distribution.

scikit-learn

scikit-learn is a free machine-learning library for Python that offers a consistent interface for pre-processing, clustering, classification and regression (Pedregosa et al., 2011). Three of its components are used in this project:

- **StandardScaler** (`sklearn.preprocessing`) rescales every feature to zero mean and unit variance.
- **KMeans** (`sklearn.cluster`) partitions the standardised records into k groups; the project uses k = 4 and `random_state=42`.

- **kneighbors_graph** (`sklearn.neighbors`) returns a sparse matrix that marks the nearest neighbours of every record; the project uses five neighbours in `connectivity` mode and excludes the record itself.

HTML, CSS and JavaScript

HTML (HyperText Markup Language) defines the structure of the page: the graph container, the sidebar with the statistics and legend, and the node detail panel. CSS (Cascading Style Sheets) controls the appearance. The project's `style.css` defines the colour variables (for example `--cluster-0` to `--cluster-3`), the dark background and a translucent "glass" panel effect created with `backdrop-filter: blur(16px)`. JavaScript adds behaviour: it calls the API with the `fetch` function, updates the statistics, builds the legend and reacts to clicks.

force-graph

force-graph is an open-source JavaScript library that draws a graph on an HTML5 canvas using a force-directed layout engine. Nodes repel one another, links pull connected nodes together, and the layout settles into a picture in which strongly connected nodes lie close together. The library provides methods such as `nodeVal`, `nodeColor`, `linkColor`, `onNodeClick`, `centerAt`, `zoom` and `zoomToFit`, all of which are used in `app.js`.

Font Awesome and Google Fonts

Font Awesome supplies the small icons used on the page (the network icon, the refresh icon on the button, and the icons in the node panel). The Outfit typeface from Google Fonts gives the interface a clean modern appearance. Both are loaded by link tags in `index.html`.

JSON and REST

JSON (JavaScript Object Notation) is a lightweight text format for exchanging structured data. The server's response is a JSON object with two arrays, `nodes` and `links`. The interface between the browser and the server follows the principles of REST (Fielding, 2000): a resource is requested through a URL using the HTTP GET method and is returned as a representation, here JSON.

4. SYSTEM DESIGN

System design converts the requirements of the analysis phase into a description of how the system will be built. It defines what enters the system, what leaves it, what data is handled, how the parts are organised and how data moves between them. All diagrams in this chapter describe the implemented workflow of the project.

4.1 INPUT DESIGN

Input design determines how data and instructions enter the system. In the current version the analyst does **not** type or upload any data. The user records are created inside the program, and the only inputs are the user's actions in the browser. Table 4.1 lists every input of the system.

Table 4.1: Inputs of the System

Input	Source	Effect
Page load	Browser opens the root address	Server sends the page; `loadNetworkData()` runs automatically
Re-Cluster click	Button in the sidebar	Requests `/api/network` again and redraws the graph
Node click	A user circle on the graph	Opens the detail panel and zooms to the node
Background click	Empty area of the canvas	Hides the detail panel
Close button click	× button of the detail panel	Hides the detail panel
Window resize	Browser window	Graph width and height are updated
Fixed parameters	Constants in `main.py`	300 users, 4 clusters, 5 neighbours, random seed 42

Because the analysis parameters are constants in the source code, they cannot be changed from the interface. Changing the number of users, clusters or neighbours currently requires editing `main.py`.

4.2 OUTPUT DESIGN

Output design decides how the results of the analysis are presented to the user. A good output is easy to read and emphasises the important information. The outputs of SocialCircle Analyzer are:

- **Network graph:** every user is a circle; the colour shows the cluster and the size grows with the number of followers; thin translucent lines show the nearest-neighbour connections.
- **Statistics:** two boxes in the sidebar show the number of users and the number of connections.
- **Cluster legend:** four coloured markers with the persona name that represents each cluster.
- **Node detail panel:** name, persona badge, followers, posts per day and likes of the selected user.
- **JSON response:** the raw output of `/api/network`, which can also be read directly in a browser.
- **Status feedback:** the Re-Cluster button changes to “Processing...” with a spinner while data is loading, and an alert message is shown if loading fails.

4.3 DATA DESIGN

The application does not use a database. All data is created in memory when a request arrives and is discarded after the response is sent. Data design therefore describes the structure of the records that exist inside the program and in the JSON response instead of database tables.

User Record (pandas DataFrame)

Table 4.2: Structure of the User DataFrame

Field	Type	Description
id	String (8 characters)	First eight characters of a random UUID
name	String	`User_1` to `User_300`
persona	String	Techie, Sports Fan, Influencer or Lurker
posts_per_day	Float	Average posts per day (not below 0)
likes_given	Float	Likes given by the user (not below 0)
followers	Float	Number of followers (not below 0)
interest_tech	Integer 0/1	Interest in technology
interest_sports	Integer 0/1	Interest in sports
interest_fashion	Integer 0/1	Interest in fashion
cluster	Integer 0–3	Added after K-Means clustering

Persona Definitions

Each user is assigned one of four personas with equal probability. The persona decides from which normal distribution the numeric values are drawn (mean and standard deviation in brackets) and how the interest flags are set. Negative draws are replaced by zero. Table 4.3 lists the definitions taken from the code.

Table 4.3: Persona Definitions Used by the Data Generator

Persona	Posts / day	Likes given	Followers	Tech	Sports	Fashion
Techie	N(5, 2)	N(20, 5)	N(300, 100)	1	0	random 0/1
Sports Fan	N(2, 1)	N(10, 3)	N(150, 50)	0	1	random 0/1
Influencer	N(10, 3)	N(50, 15)	N(5000, 1000)	random	random	1
Lurker	N(0.5, 0.5)	N(40, 10)	N(50, 20)	random	random	random

JSON Response Structure

Table 4.4: Fields of a Node Object

Field	Type	Meaning
id	String	Unique user identifier
name	String	Display name shown as the node label
val	Number	Node size = 10 + followers / 500
cluster	Integer 0–3	Cluster assigned by K-Means
persona	String	Persona used when the record was generated
posts	Number	Posts per day
likes	Number	Likes given
followers	Number	Followers

Table 4.5: Fields of a Link Object

Field	Type	Meaning
source	String	Identifier of the user whose neighbour is listed
target	String	Identifier of one of the five nearest neighbours

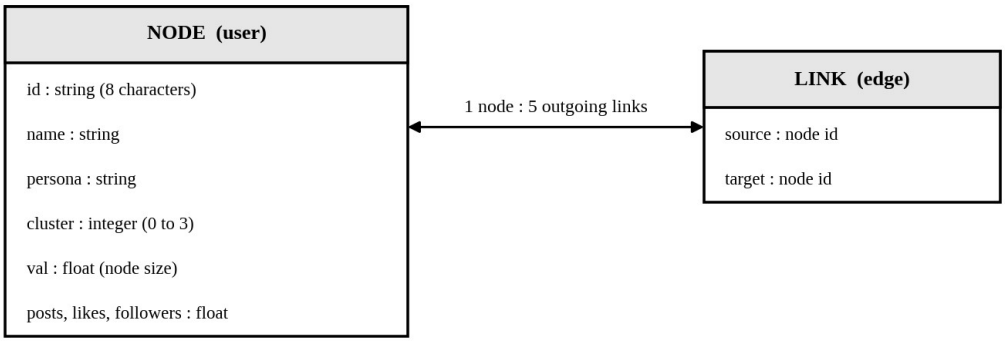


Figure 4.1: Logical Data Model of the JSON Response

4.4 SYSTEM ARCHITECTURE

SocialCircle Analyzer follows a three-tier organisation. The **presentation tier** is the web browser, which holds the HTML page, the style sheet and the JavaScript that calls the API and draws the graph. The **application tier** is the FastAPI server running on Uvicorn; it serves the static files and exposes the `/api/network` endpoint. The **analysis tier** consists of the functions that generate data, standardise features, cluster users and build the graph. All three tiers run inside the same Python process, and no external database is involved. Figure 4.2 shows the architecture.

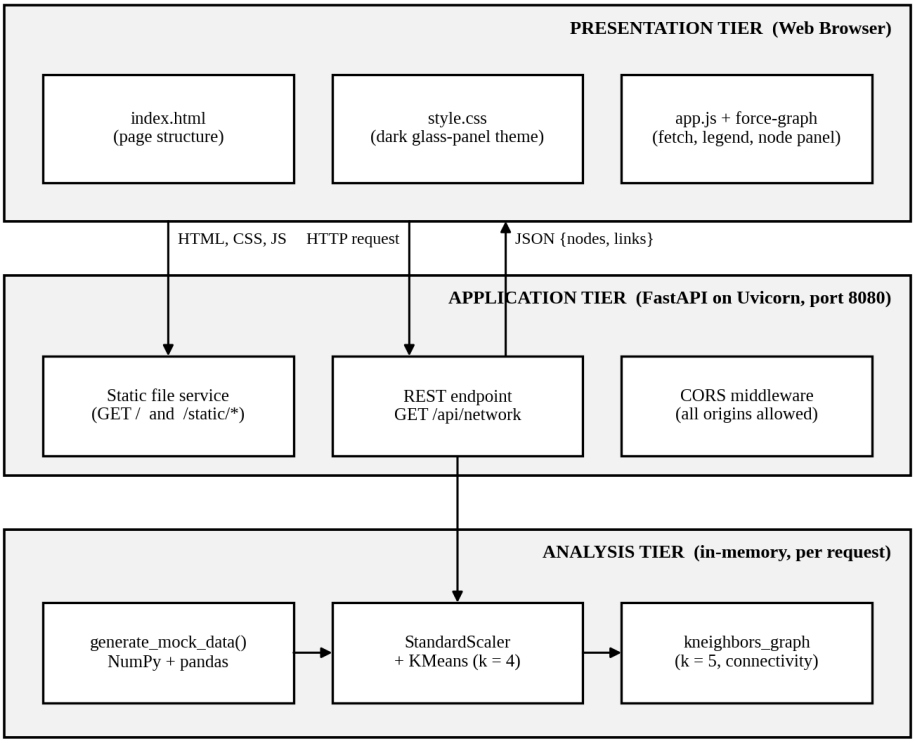


Figure 4.2: System Architecture of SocialCircle Analyzer

4.5 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system. It shows the processes that transform data, the stores in which data rests, the external entities that supply or receive data, and the flows between them. A DFD does not show control logic or timing. Four symbols are used to construct a DFD, shown in Figure 4.3.

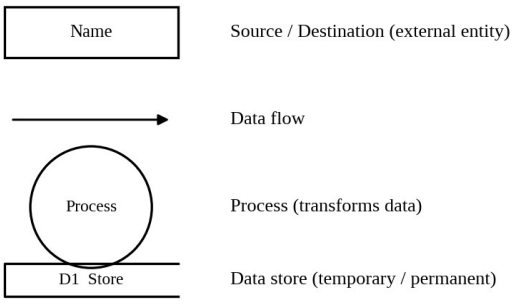


Figure 4.3: Data Flow Diagram Notation

Level 0 DFD (Context Diagram)

The context diagram treats the whole system as a single process. The only external entity is the user (analyst). The user's page loads, Re-Cluster clicks and node clicks enter the system, and the network graph, statistics, legend and node details leave it.

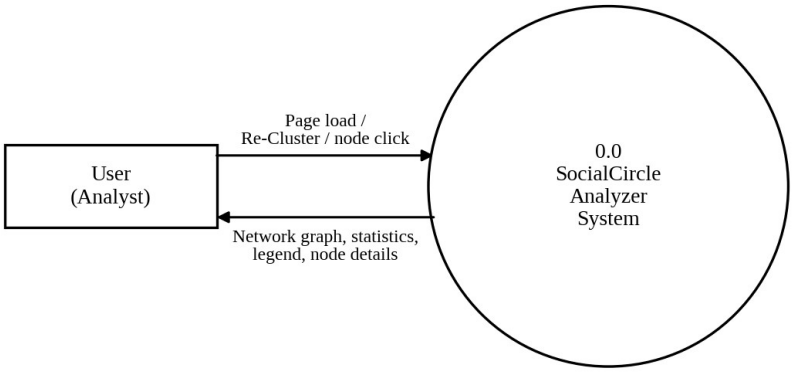


Figure 4.4: Level 0 DFD – Context Diagram

Level 1 DFD

At the next level the system is divided into five processes. Process 1.0 serves the web interface from the static files (store D1). Process 4.0 is the browser-side application: it receives user actions, requests the analysis and draws the graph. Process 2.0 generates the user records and keeps them in the in-memory DataFrame (store D2). Process 3.0 clusters the users and builds the graph and passes nodes, links and cluster labels back to process 4.0. Process 5.0 shows the details of a clicked node.

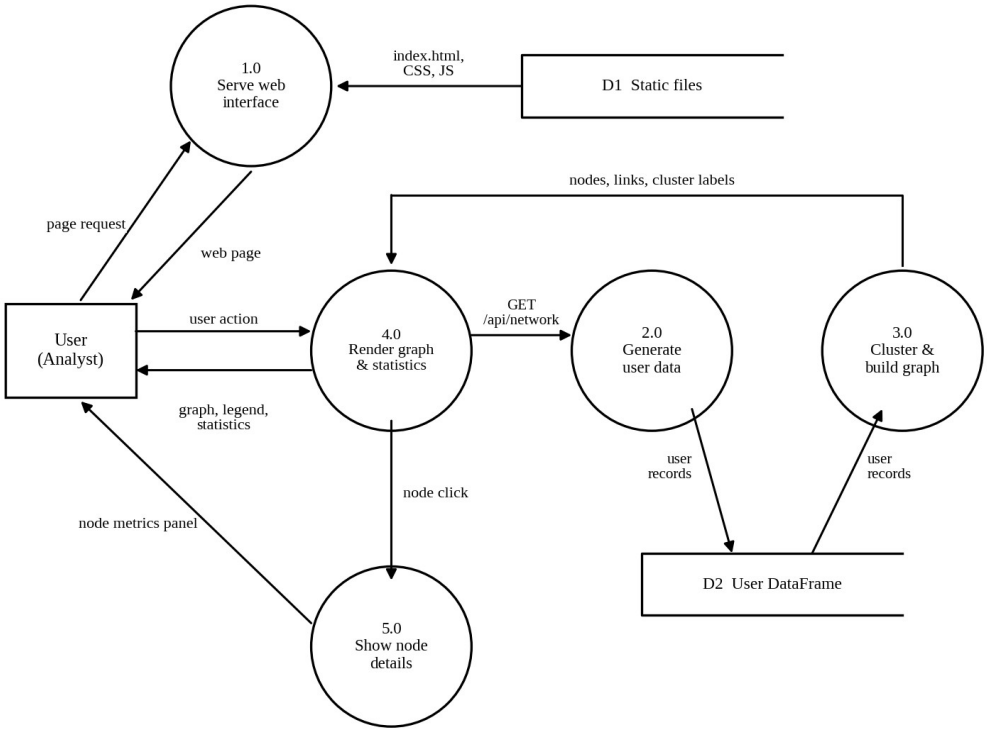


Figure 4.5: Level 1 DFD of SocialCircle Analyzer

Level 2 DFD – Process 3.0

Process 3.0 is expanded in Figure 4.6. The six features are selected from the DataFrame, standardised, and used in two ways: K-Means produces the cluster labels, and the nearest-neighbour search produces the sparse adjacency matrix. Both results are assembled into the JSON structure of nodes and links.

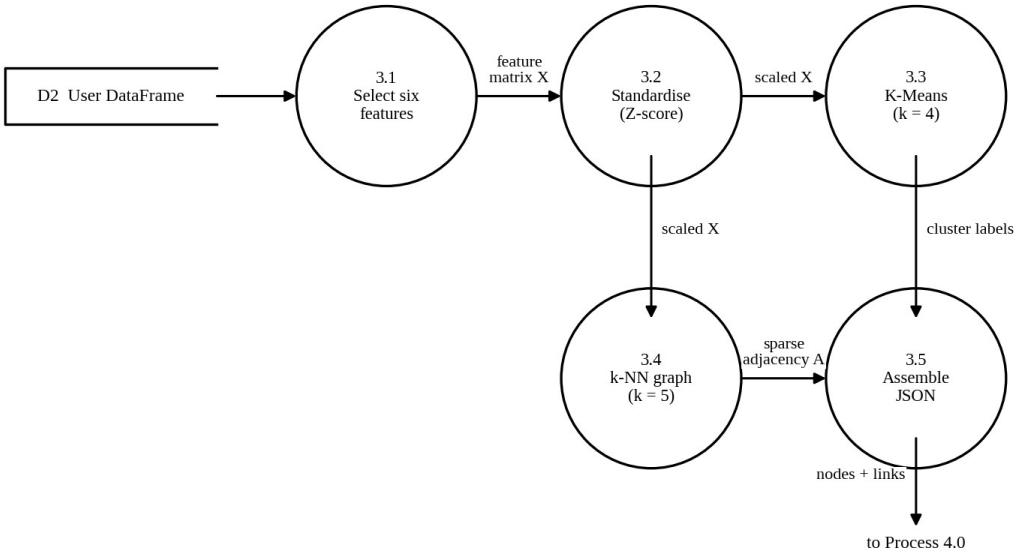


Figure 4.6: Level 2 DFD – Clustering and Graph Construction

Data Processing Flow

Figure 4.7 follows the shape of the data as it moves through the pipeline of the `/api/network` function. The raw records have nine columns; six of them form the feature matrix; the matrix is standardised; the labels and the graph are computed from it; and both are converted to JSON.

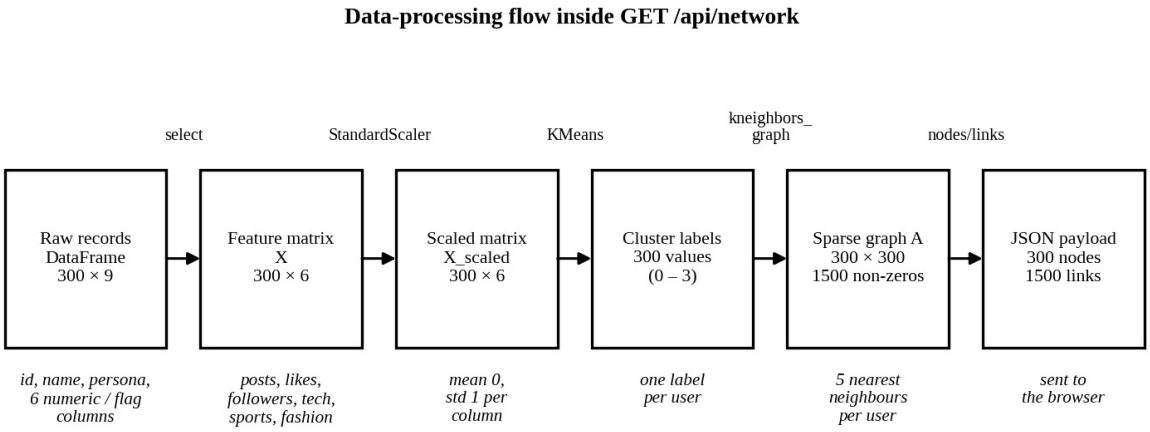


Figure 4.7: Data-Processing Flow inside the API Function

Process Specification

Table 4.6 describes each process of the data flow diagrams in terms of its input, its logic and its output, as implemented in the source code. It complements the diagrams by stating exactly which function performs each step.

Table 4.6: Process Specification

Process	Input	Processing logic	Output
1.0 Serve web interface	HTTP request for `/` or `/static/*`	`serve_index()` returns `index.html`; `StaticFiles` returns the other files	HTML, CSS and JavaScript files
2.0 Generate user data	Number of users (300)	`generate_mock_data()` seeds the random generators, chooses a persona per user and draws values from normal distributions, clipping negatives to zero	DataFrame of 300 records with 9 columns
3.1 Select features	DataFrame	Selects the six columns used for clustering	300 × 6 feature matrix
3.2 Standardise	Feature matrix	`StandardScaler.fit_transform` gives each column mean 0 and standard deviation 1	Scaled matrix
3.3 K-Means	Scaled matrix	`KMeans(n_clusters=4, random_state=42).fit_predict`	300 cluster labels (0 to 3)
3.4 k-NN graph	Scaled matrix	`kneighbors_graph(n_neighbors=5, mode="connectivity", include_self=False)`	Sparse 300 × 300 matrix
3.5 Assemble JSON	Labels, matrix, DataFrame	Builds one node dictionary per row (including `val = 10 + followers / 500`) and one link per non-zero matrix entry	Dictionary with `nodes` and `links`
4.0 Render graph and statistics	JSON response	`Graph.graphData`, statistics update, `createLegend`, `zoomToFit`	Interactive graph, counts, legend
5.0 Show node details	Clicked node object	`showNodePanel(node)` fills and reveals the panel; camera moves to the node	Detail panel

4.6 USE CASE DIAGRAM

A use case diagram shows how an actor interacts with the system. The system has a single actor, the user (analyst), because there is no login or administrator function. The use cases correspond to the actions available in the interface. Selecting a user node includes viewing that user’s metrics panel.

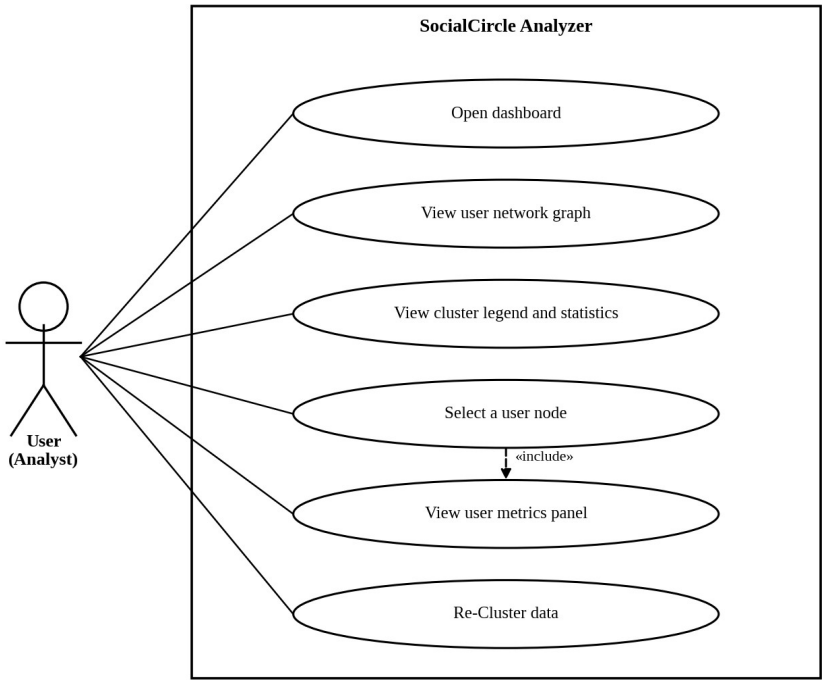


Figure 4.8: Use Case Diagram

4.7 ACTIVITY DIAGRAM

The activity diagram in Figure 4.9 follows the workflow from opening the page to the user actions that are possible afterwards. The steps between the request and the rendering are performed automatically by the server and the browser; after the first rendering the user can select a node, hide the panel, or start the analysis again.

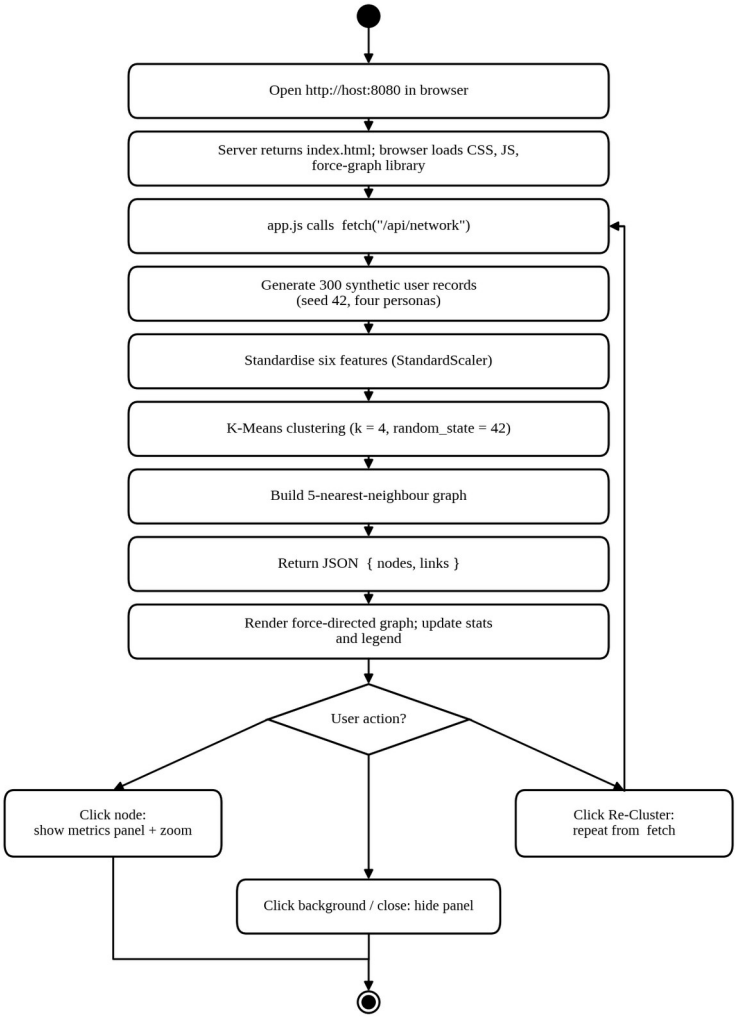


Figure 4.9: Activity Diagram of the Overall Workflow

4.8 SEQUENCE DIAGRAM

The sequence diagram in Figure 4.10 shows the order of messages between the user, the browser script, the FastAPI server, the analysis functions and the force-graph library when the page is opened and a node is clicked.

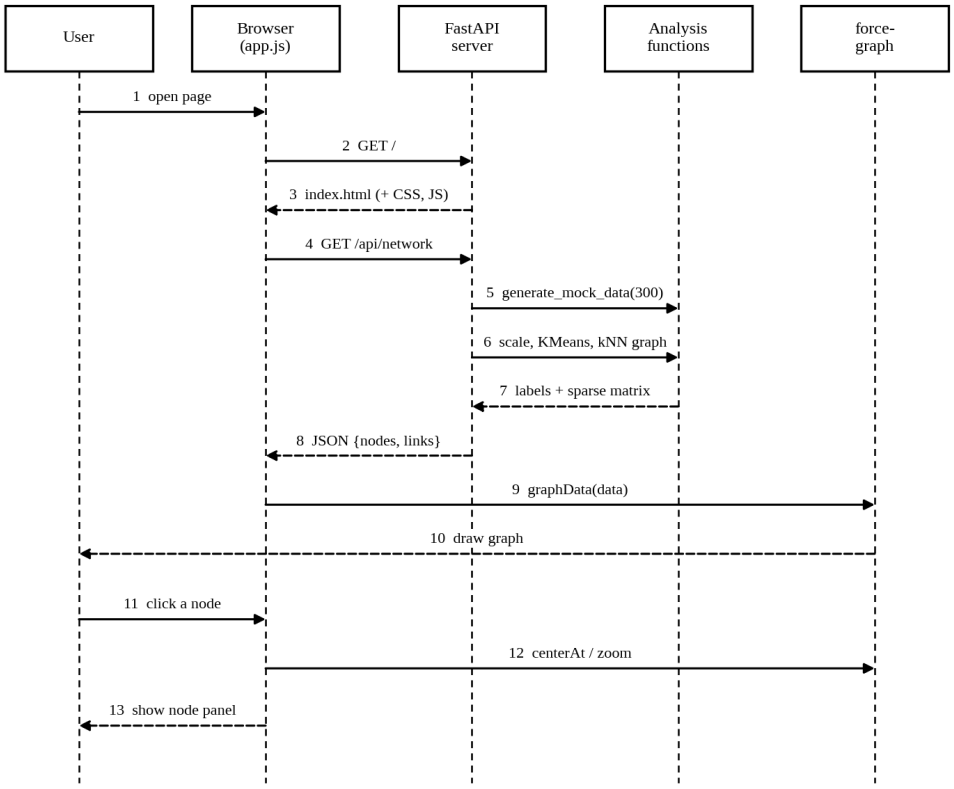


Figure 4.10: Sequence Diagram of Page Load and Node Click

4.9 USER INTERFACE DESIGN

The interface consists of a full-screen graph canvas with two floating panels. The design choices, all of which can be verified in `style.css` and `index.html`, are described below.

- **Canvas:** the graph fills the whole window on a very dark background (`#0b0f19`).
- **Sidebar:** a 320-pixel wide translucent glass panel in the upper-left corner contains the brand name, the sub-title, the statistics boxes, the cluster legend and the Re-Cluster button.
- **Node panel:** a second glass panel, hidden by default, appears when a node is clicked and shows the avatar (coloured like the cluster), the name, the persona badge and three metrics with icons.
- **Colour palette:** cluster 0 rose (`#f43f5e`), cluster 1 blue (`#3b82f6`), cluster 2 emerald (`#10b981`) and cluster 3 violet (`#8b5cf6`).
- **Typography and icons:** Outfit typeface and Font Awesome icons.

Table 4.7: Main Visual Elements Defined in style.css

Element	Purpose	Design detail
CSS variables (`:root`)	One place for colours	`--cluster-0` to `--cluster-3` hold the four cluster colours
Body / canvas	Full-screen graph area	Very dark background (`#0b0f19`), Outfit typeface
Sidebar panel	Title, statistics, legend, button	Fixed position, 320 px width, blurred translucent background
Statistics boxes	Users and Connections counts	Two equal boxes with large numerals
Legend items	Colour key for clusters	Small coloured circle followed by the persona name
Re-Cluster button	Repeat the analysis	Gradient button with hover effect; icon spins while loading
Node panel	Details of a selected user	Hidden by default (`.hidden`), avatar, badge and three metrics

The style sheet contains no media queries, so the layout is designed for desktop-sized screens and has not been adapted separately for phones.

5. SYSTEM SPECIFICATION

5.1 MODULE DESCRIPTION

A module is a self-contained part of the system that performs one clearly defined task. SocialCircle Analyzer is divided into eight modules: five on the server side (all inside `backend/main.py`) and three on the browser side (inside `static/app.js`, with `index.html` and `style.css` providing structure and appearance). Figure 5.1 shows how the modules are related.

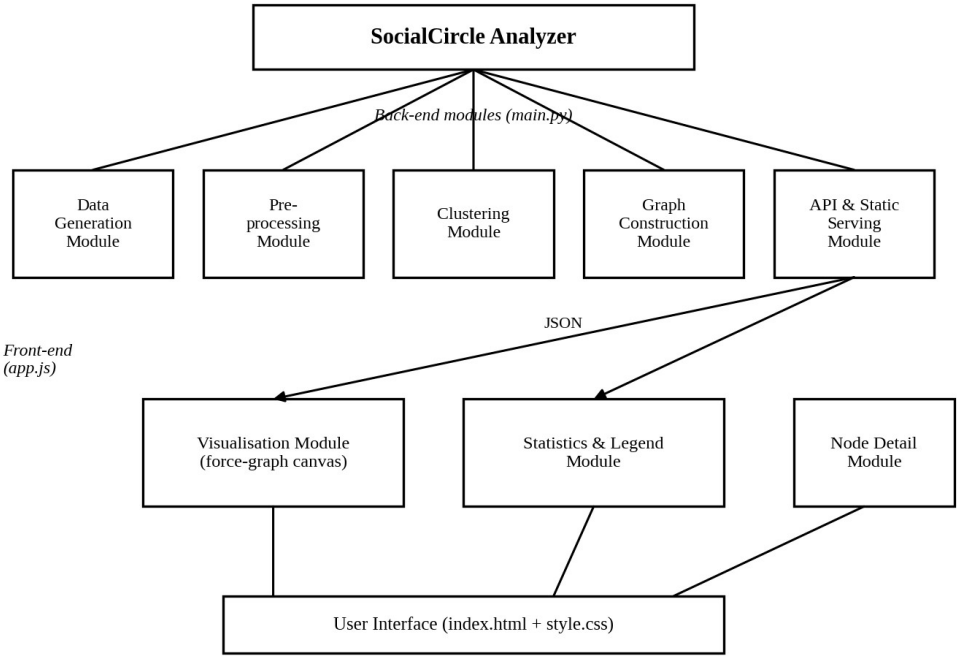


Figure 5.1: Module Diagram of SocialCircle Analyzer

DATA GENERATION MODULE

This module is implemented by the function `generate_mock_data(n_users=200)`. The API calls it with `n_users = 300`. It fixes the NumPy and Python random seeds to 42, then repeats the following steps for every user: choose one of the four personas at random, draw the number of posts per day, likes given and followers from the normal distribution defined for that persona, set the three interest flags, create an eight-character identifier from a random UUID, and name the user `User_1`, `User_2` and so on. Negative values are replaced by zero with `max(0, value)`. The module returns a pandas `DataFrame`.

PRE-PROCESSING MODULE

The first part of `get_network_data()` selects the six clustering features (`posts_per_day`, `likes_given`, `followers`, `interest_tech`, `interest_sports`, `interest_fashion`) and standardises them with `StandardScaler`. Standardisation is necessary because the features are on very different scales: followers range from about 6 to about 7,600 while the interest flags are only 0 or 1. Without it, followers would dominate the distance calculation.

CLUSTERING MODULE

The standardised matrix is passed to `KMeans(n_clusters=4, random_state=42)`. The `fit_predict` call returns one label from 0 to 3 for every user, and the labels are stored in a new `cluster` column of the DataFrame.

GRAPH CONSTRUCTION MODULE

The same standardised matrix is passed to `kneighbors_graph(X_scaled, n_neighbors=5, mode="connectivity", include_self=False)`. The result is a sparse 300 × 300 matrix in which entry (u, v) is 1 when v is one of the five nearest neighbours of u. The function then reads the non-zero positions of the matrix and creates one link object (`source`, `target`) for each, giving 300 × 5 = 1,500 links.

API AND STATIC SERVING MODULE

This module creates the `FastAPI` application, adds the CORS middleware, defines the `GET /api/network` endpoint, mounts the `../static` folder at `/static`, and defines the root route `GET /` which returns `index.html` as a `FileResponse`. When the file is run directly, Uvicorn starts the server on port 8080.

VISUALISATION MODULE

In `app.js` the `ForceGraph()` object is created on the `graph-container` element. It is configured with a background colour, `nodeVal("val")` for node size, `nodeLabel("name")` for the hover label, a colour function that maps `node.cluster` to the palette read from the CSS variables, and a faint white link colour. After new data is received, `Graph.graphData(data)` draws the network and `Graph.zoomToFit()` scales it to the window.

STATISTICS AND LEGEND MODULE

After each successful response `loadNetworkData()` writes `data.nodes.length` and `data.links.length` into the two statistic boxes. `createLegend(nodes)` builds one legend entry per cluster: it walks through the nodes and, for each cluster number, remembers the persona of the first node it meets, then displays that persona name beside the cluster colour. The legend therefore names each cluster after the first persona encountered in it; this is a simple labelling rule and not a measured cluster property (see Section 6.9).

NODE DETAIL MODULE

`showNodePanel(node)` reveals the hidden panel and fills in the name, the persona badge, the rounded follower count, the posts per day (one decimal place) and the rounded likes. The avatar circle is coloured with the cluster colour. `hideNodePanel()` hides the panel again. The click handlers also move the camera to the node with `centerAt` and `zoom(4, 2000)`.

5.2 ALGORITHMS AND TECHNIQUES

Only techniques that are present in the source code are described in this section. The project uses classical data-analysis methods; it does not train a predictive model, and no accuracy figure is reported because the data has no ground-truth labels for the algorithm to predict.

Feature Standardisation (Z-score)

`StandardScaler` transforms each feature column independently so that its mean becomes zero and its standard deviation becomes one:

$$z = (x - \mu) / \sigma$$

where x is the original value, μ is the mean of the column and σ is its standard deviation. After this step every feature has equal influence on Euclidean distances.

K-Means Clustering

K-Means (MacQueen, 1967; Lloyd, 1982) divides n points into k clusters so that each point belongs to the cluster with the nearest centre. The algorithm minimises the sum of squared distances between the points and the centres of their clusters:

$$J = \Sigma\ (i = 1 \text{ to } k)\ \ \Sigma\ (x \text{ in cluster } Ci)\ \ \| x - \mu i \|^2$$

where μi is the mean of the points in cluster Ci. The procedure, illustrated in Figure 5.2, is:

1. Choose k initial centres. scikit-learn uses the k-means++ method by default (Arthur and Vassilvitskii, 2007), which spreads the initial centres apart.
2. Assign every point to the nearest centre.
3. Recompute every centre as the mean of the points assigned to it.
4. Repeat steps 2 and 3 until the centres no longer change significantly.

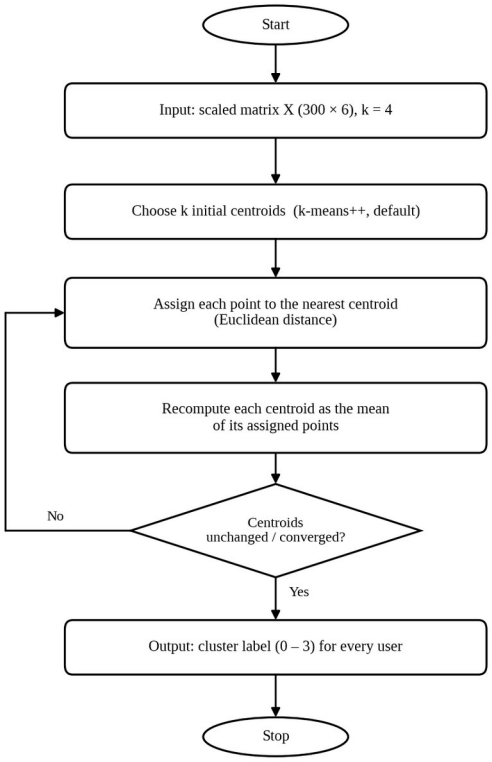


Figure 5.2: Flowchart of the K-Means Procedure

In this project k = 4 is a fixed choice that matches the four personas used by the data generator. The code does not test other values of k. The parameter `random_state=42` makes the result repeatable.

k-Nearest-Neighbour Graph

The nearest-neighbour idea (Cover and Hart, 1967) is used here to build a graph rather than to classify. For each user the five users with the smallest Euclidean distance in the standardised six-dimensional space are found, and a link is drawn from the user to each of them. Because “nearest” is not symmetric, the graph is directed in principle: if B is among the five nearest neighbours of A, A need not be among those of B. In the measured run, 528 pairs of users were linked in both directions. The browser draws the links as plain lines.

Node Size Rule

The size value passed to the graph is calculated on the server as:

$$val = 10 + (followers / 500)$$

The constant 10 ensures that every node is visible, and dividing by 500 keeps the largest accounts from becoming too large. In the measured run the value ranged from about 10.0 to about 25.3.

Force-Directed Layout

The position of every node on the screen is decided by the force-graph library. In force-directed methods (Fruchterman and Reingold, 1991) nodes act like charged particles that repel each other while links act like springs that pull connected nodes together. When the simulation settles, densely connected users lie close together. The positions are computed in the browser and are not part of the server response.

5.3 WORKED EXAMPLE OF THE ANALYSIS

To make the algorithms of the previous section concrete, this section follows two users of the generated data set through the pipeline. All numbers were produced by running the submitted code (scikit-learn 1.8.0, seed 42) and are rounded to two decimals.

Step 1: Statistics of the Features

Table 5.1 gives the mean and standard deviation that `StandardScaler` calculates for each of the six features from the 300 records (population standard deviation, as used by scikit-learn), together with the smallest and largest values. The very different scales of the columns – followers up to about 7,632 against flags of only 0 or 1 – show why standardisation is required.

Table 5.1: Statistics of the Six Clustering Features (300 users)

Feature	Mean	Std. deviation	Minimum	Maximum
Posts per day	4.74	4.14	0.00	17.72
Likes given	29.83	16.98	3.79	84.06
Followers	1,560.97	2,298.86	5.78	7,632.38
Interest in tech	0.52	0.50	0.00	1.00
Interest in sports	0.48	0.50	0.00	1.00
Interest in fashion	0.63	0.48	0.00	1.00

Step 2: Standardising a User

The first two generated users are User_1 (a Techie) and User_2 (an Influencer). Applying $z = (x - \mu) / \sigma$ to their raw values gives the standardised values in Table 5.2. For example, the posts per day of User_1 are (5.99 − 4.74) / 4.14 = +0.30, i.e. slightly above the average. The followers of User_2 are (4,765.86 − 1,560.97) / 2,298.86 = +1.39, i.e. about one and a half standard deviations above the average.

Table 5.2: Raw and Standardised Values of User_1 and User_2

Feature	User_1 raw	User_1 z	User_2 raw	User_2 z
Posts per day	5.99	+0.30	14.57	+2.37
Likes given	19.31	-0.62	46.49	+0.98
Followers	364.77	-0.52	4765.86	+1.39
Interest in tech	1.0	+0.95	0.0	-1.05
Interest in sports	0.0	-0.95	0.0	-0.95
Interest in fashion	0.0	-1.31	1.0	+0.76

Step 3: Cluster Centres and Assignment

After the K-Means iterations have converged, each cluster is represented by its centre, which is the mean of its members in standardised space. Table 5.3 shows the four centres found in the test run. Positive values mean that the cluster is above the average of all users in that feature.

Table 5.3: Cluster Centres in Standardised Feature Space

Cluster	Posts	Likes	Followers	Tech	Sports	Fashion
0	+1.28	+0.92	+1.56	-0.05	-0.00	+0.76
1	-0.69	-1.07	-0.62	-1.05	+1.05	-0.15
2	-0.03	-0.52	-0.57	+0.95	-0.95	-0.30
3	-1.01	+0.83	-0.65	-0.14	+0.21	-0.48

The centres are consistent with the cluster profiles reported in Chapter 6: cluster 0 is high in posts, likes and followers and in the fashion flag; cluster 1 is defined mainly by the sports flag (+1.05) and low activity; cluster 2 by the tech flag (+0.95); and cluster 3 by high likes together with low posts and followers.

A user is assigned to the cluster whose centre is nearest in Euclidean distance. The distances of the two example users to the four centres are given in Table 5.4. User_1 is nearest

to the centre of cluster 2 (distance 1.07) and User_2 to the centre of cluster 0 (distance 1.77). These are the clusters that the code returned for them. Over all 300 users the mean distance to the own cluster centre is 1.46.

Table 5.4: Distance of the Example Users to the Cluster Centres

User	Cluster 0	Cluster 1	Cluster 2	Cluster 3	Assigned
User_1 (Techie)	3.72	3.25	1.07	2.66	2
User_2 (Influencer)	1.77	4.74	4.13	4.4	0

Step 4: Neighbours and Node Size

The five nearest neighbours of each user are found among all other 299 users in the same standardised space, and each pair becomes a link. The node size of User_2 is 10 + 4,765.86 / 500 = 19.53, while that of User_1 is 10 + 364.77 / 500 = 10.73, so the Influencer is drawn almost twice as large as the Techie.

5.4 API SPECIFICATION

Table 5.5: HTTP Endpoints of the Application

Method and path	Purpose	Response
GET /	Serves the web page	`index.html` (HTML)
GET /static/{file}	Serves `app.js`, `style.css` and other static files	File content
GET /api/network	Runs the analysis on 300 generated users	JSON object with `nodes` and `links`

The endpoint takes no parameters and requires no authentication. Because FastAPI is used, an automatically generated interactive documentation page is also available at `/docs` when the server is running; this is a standard FastAPI feature and is not part of the project code.

5.5 FRONT-END FUNCTION SPECIFICATION

Table 5.6: Functions and Handlers in app.js

Name	Type	Description
`colorPalette`	Object	Reads `--cluster-0` to `--cluster-3` from the CSS and maps cluster numbers to colours

Name	Type	Description
`ForceGraph()` setup	Initialisation	Creates the graph and registers node and background click handlers
`loadNetworkData()`	Async function	Shows a spinner, fetches `/api/network`, updates the graph, statistics and legend, then fits the graph to the window
`createLegend(nodes)`	Function	Builds the legend items from the clusters present in the data
`showNodePanel(node)`	Function	Fills and reveals the node detail panel
`hideNodePanel()`	Function	Hides the node detail panel
Close-button listener	Event handler	Calls `hideNodePanel()`
Re-Cluster listener	Event handler	Calls `loadNetworkData()`
Resize listener	Event handler	Sets the graph width and height to the window size

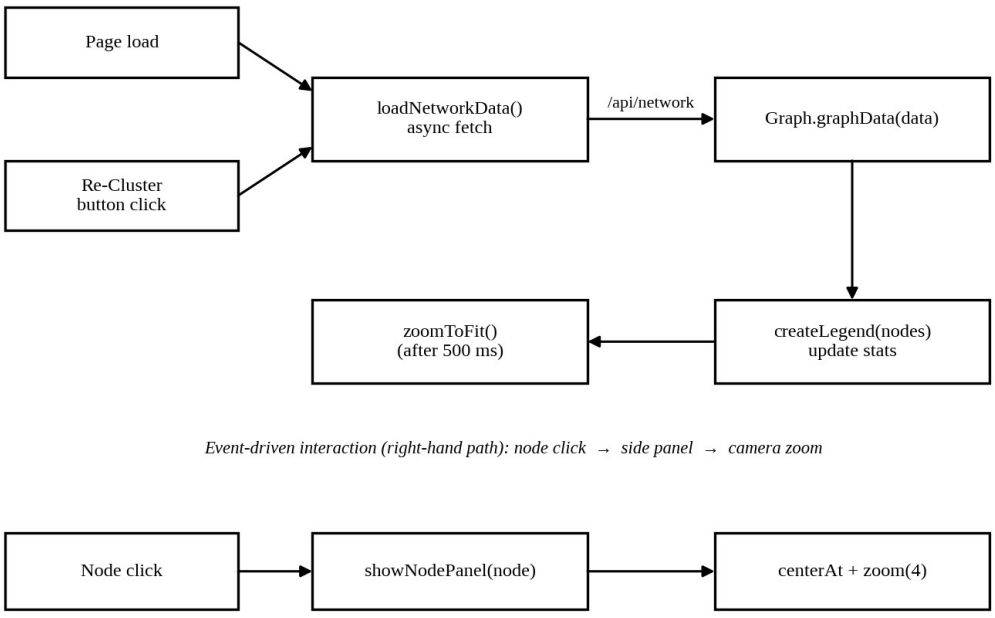


Figure 5.3: Event Flow of the Front-End Script

6. SYSTEM IMPLEMENTATION

Implementation is the stage of the project in which the theoretical design is turned into a working system. It converts the design into program code, installs the software and puts the system into an operational state. This chapter describes the project files, the implementation of the back end and the front end, the steps for running the application, the error handling present in the code, and the results obtained by running the actual backend functions.

6.1 PROJECT STRUCTURE

Table 6.1: Files of the Project

File	Lines	Purpose
`backend/main.py`	142	FastAPI application, data generation, clustering, graph construction, server start
`backend/requirements.txt`	7	List of Python packages
`static/index.html`	89	Page structure: graph container, sidebar, node panel, external resources
`static/app.js`	114	Graph setup, API call, legend, node panel and events
`static/style.css`	271	Dark theme, glass panels, layout, cluster colours

The folder `backend/venv` holds the Python virtual environment created for the project and is not part of the source code. A file `__pycache__/main.cpython-312.pyc` shows that the project was run with Python 3.12.

6.2 BACK-END IMPLEMENTATION

Application Set-up

The application object is created with `app = FastAPI()`. The CORS middleware is added with `allow_origins=["*"]`, which lets pages from any address call the API. This is convenient for development but should be restricted in a real deployment.

Data Generation

The generator uses a loop over the requested number of users. The code below shows the branch for the Influencer persona, which has the largest values for posts, likes and followers:

```
elif persona == 'Influencer':
    posts = np.random.normal(10, 3)
```

```
    likes = np.random.normal(50, 15)
    followers = np.random.normal(5000, 1000)
    tech = random.choice([0, 1])
    sports = random.choice([0, 1])
    fashion = 1
```

Clustering and Graph Construction

The endpoint function applies the pipeline in a few lines of code. The essential statements are:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
df['cluster'] = clusters

A = kneighbors_graph(X_scaled, n_neighbors=5,
                     mode='connectivity', include_self=False)
```

Building the JSON Response

The function loops over the DataFrame rows to create the node dictionaries, converts NumPy values to standard Python types with `int()` and `float()` so that they can be serialised, and loops over the non-zero entries of the sparse matrix to create the link dictionaries. FastAPI then converts the returned dictionary to JSON automatically.

Serving the Front End

`app.mount("/static", StaticFiles(directory="../static"), name="static")` makes the front-end files available, and the route `@app.get("/")` returns `../static/index.html`. Because the paths are relative (`../static`), the server has to be started from inside the `backend` folder.

6.3 FRONT-END IMPLEMENTATION

`index.html` defines a full-screen `div` for the graph, the sidebar panel with the brand, the two statistic boxes, the legend container and the Re-Cluster button, and the hidden node panel. It loads the force-graph library from unpkg, Font Awesome from cdnjs and the Outfit font from Google Fonts.

`app.js` reads the four cluster colours from the CSS variables so that the palette is defined in one place. The graph is created and configured as a chain of method calls. `loadNetworkData()` disables the button, replaces its label with a spinner and the text “Processing...”, awaits the response, passes the data to the graph, updates the statistics, builds the legend and re-enables the button.

`style.css` implements the visual design: CSS variables for the palette, the translucent glass effect, the fixed positions of the two panels, the hover effects of the button and the small badge and metric components of the node panel.

6.4 RUNNING THE APPLICATION

The application is installed and started with the following steps. They are derived from the project layout and were not run inside a browser during the preparation of this report.

1. Install Python 3.12 and open a command prompt in the project folder.
2. Go to the backend folder: `cd backend`.
3. Create and activate a virtual environment: `python -m venv venv` and then `venv\Scripts\activate` (Windows).
4. Install the packages: `pip install -r requirements.txt`.
5. Start the server: `python main.py`. Uvicorn listens on port 8080. (`uvicorn main:app --reload --port 8080` may also be used.)
6. Open a browser and go to `http://localhost:8080`. The graph is drawn automatically.

6.5 SOURCE CODE WALKTHROUGH

The modules imported by `main.py` and the way each is used are summarised in Table 6.2. The table is a reading of the source code and can be used as a checklist when the code is explained during the viva-voce.

Table 6.2: Imports of main.py and Their Use

Import	Used for
`FastAPI`	Creating the application object `app`
`CORSMiddleware`	Adding the cross-origin middleware to `app`
`numpy as np`	Random number generation (`np.random.seed`, `np.random.normal`)
`pandas as pd`	Creating the `DataFrame` of users
`KMeans`	Clustering the standardised users
`kneighbors_graph`	Building the nearest-neighbour matrix
`StandardScaler`	Standardising the six features
`random`	Seeding and choosing personas and interest flags
`uuid`	Creating the eight-character user identifiers

Import	Used for
`os`	Imported at the top of the file
`StaticFiles`, `FileResponse`	Serving `/static` and the home page
`uvicorn`	Imported inside the `__main__` block to start the server

The front-end script `app.js` is organised in the order in which it runs: the colour palette, the graph object, the data-loading function, the legend function, the node-panel functions and finally the event listeners. The graph object is configured through chained calls, for example `.nodeVal("val")` for the size, `.nodeLabel("name")` for the label, `.linkColor(...)` for the translucent links and `.onNodeClick(...)` and `.onBackgroundClick(...)` for the two click events. Since the script is only 114 lines long, the whole flow of the application can be followed from the first line to the last.

6.6 ERROR HANDLING

The error handling present in the code is modest and is described here exactly. In `loadNetworkData()` the fetch call is enclosed in a `try ... catch` block. If the request fails, the error is written to the browser console and an alert “Failed to load data. Is the backend running?” is shown. The backend has no explicit exception handling; if an error occurs, FastAPI returns its standard HTTP 500 response. The code does not check the HTTP status of the response before parsing it. It was also observed in the code that the catch block does not restore the button, so after a failure the button remains disabled with the text “Processing...” until the page is reloaded. This is recorded as a limitation in Section 6.9.

6.7 RESULTS AND DISCUSSION

The results below were obtained by executing the functions `generate_mock_data()` and `get_network_data()` from the submitted `main.py` in Python 3.12.3 with scikit-learn 1.8.0, pandas 3.0.2 and NumPy 2.4.4. Because the data is generated with a fixed seed, the numbers below are repeatable. Slightly different package versions (the project’s own environment has scikit-learn 1.9.1) may change the numbers slightly. The values are not test-set accuracies; they describe the behaviour of the clustering on this generated data.

Size of the Output

Table 6.3: Size of the API Result

Quantity	Value
Nodes (users)	300
Links (connections)	1,500 (300 users × 5 neighbours)
Links between users of the same cluster	1,481 (98.7 %)
Pairs of users linked in both directions	528
Response size	126,750 bytes
K-Means iterations until convergence	7
K-Means inertia (sum of squared distances)	679.28

Cluster Composition

Table 6.4 compares the cluster labels found by K-Means with the personas that were used to generate the data. The personas are not shown to the algorithm; it only sees the six numeric features.

Table 6.4: Cluster Composition by Generated Persona

Cluster	Influencer	Sports Fan	Techie	Lurker	Total
0	84	0	0	0	84
1	0	67	0	4	71
2	0	0	75	15	90
3	0	0	0	55	55
Total	84	67	75	74	300

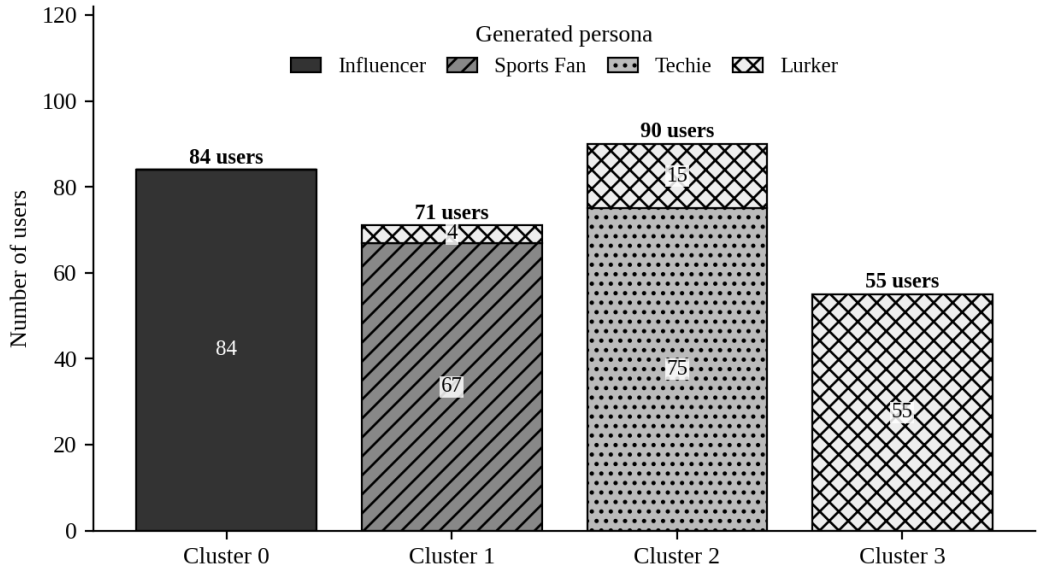


Figure 6.1: Cluster Composition by Generated Persona

The table shows that the Influencers (cluster 0), Sports Fans (cluster 1, apart from four Lurkers) and Techies (cluster 2, apart from fifteen Lurkers) are separated cleanly, and that 55 of the 74 Lurkers form cluster 3. The remaining 19 Lurkers were placed with the Sports Fan and Techie clusters. A likely explanation, which was not tested separately, is that Lurkers have random interest flags: a Lurker whose flags happen to be (tech = 1, sports = 0) resembles a Techie in those columns, and the low activity values of Techies and Lurkers are not far apart after standardisation.

Cluster Profiles

Table 6.5: Mean Feature Values of Each Cluster

Cluster	Users	Posts per day	Likes given	Followers
0	84	10.04	45.44	5,148.5
1	71	1.89	11.64	143.1
2	90	4.60	20.98	250.1
3	55	0.58	43.96	57.3

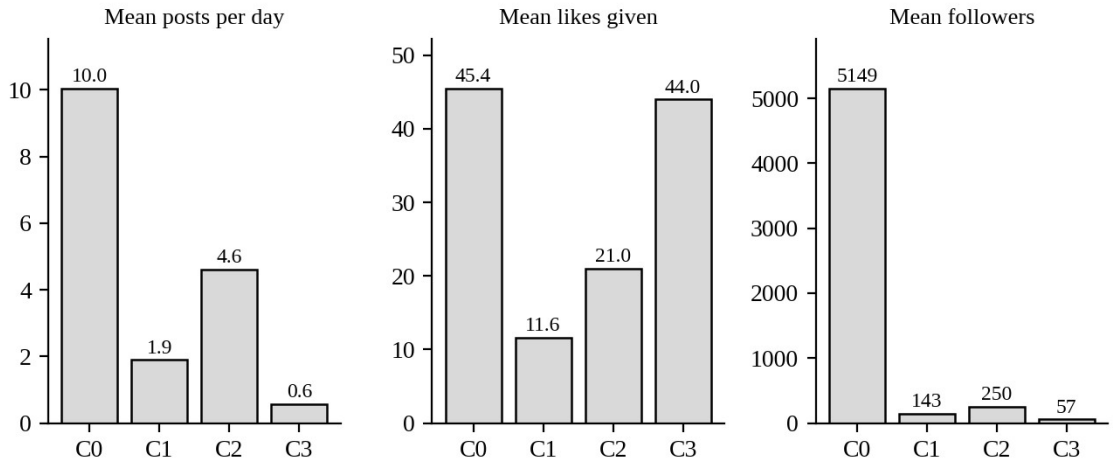


Figure 6.2: Mean Posts, Likes and Followers of Each Cluster

The profiles are consistent with the persona definitions in Table 4.3. Cluster 0 has by far the most followers and posts; cluster 3 posts least and has the fewest followers, but gives almost as many likes as cluster 0 (43.96 against 45.44), which is the typical behaviour of a Lurker; clusters 1 and 2 have low and medium activity respectively.

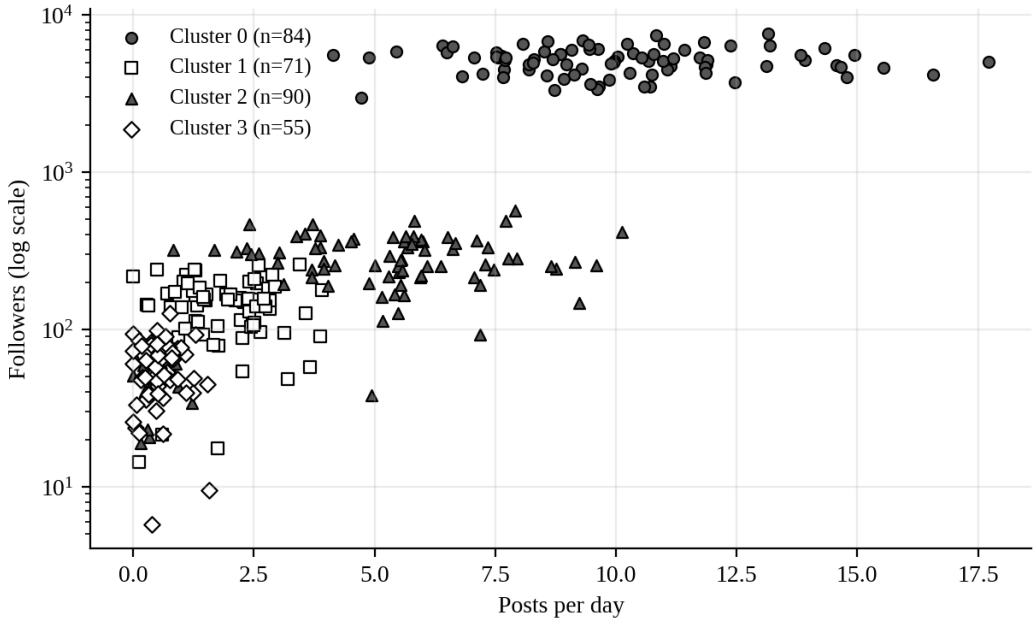


Figure 6.3: Followers (log scale) against Posts per Day, Coloured by Cluster

Interest Flags by Persona

The three interest columns are binary flags. Table 6.6 gives their average value for each generated persona; it is the share of users of that persona who have the flag set. The values agree with the generator rules of Table 4.3: all Techies have the tech flag, all Sports

Fans the sports flag and all Influencers the fashion flag, while the remaining flags are random with a probability of about one half.

Table 6.6: Share of Users with Each Interest Flag, by Persona

Persona	Tech	Sports	Fashion
Influencer	0.50	0.48	1.00
Sports Fan	0.00	1.00	0.54
Techie	1.00	0.00	0.47
Lurker	0.54	0.49	0.47

These shares explain the shape of the cluster centres. The interest flags carry as much weight in the distance calculation as the activity measures, which is why the clusters separate Techies and Sports Fans mainly by their interest flag.

Network Structure

Since 98.7 % of the links join users of the same cluster, the force-directed layout places every cluster as a visibly separate group, and the four colours can be seen as four regions of the graph. Only 19 links out of 1,500 cross between clusters. This is an expected result of building the graph from the same standardised features that were used for clustering, and it should not be read as evidence of real social ties: the links express similarity of behaviour in the generated data, not communication between people.

The graph view, the legend, the node detail panel and the JSON output are reproduced in the screen shots of Appendix I (Figures Figure 11.1 to Figure 11.7).

6.8 ADVANTAGES OF THE SYSTEM

- Groups users using all six features at once instead of one column at a time.
- Shows clusters and similarity links in a single interactive picture.
- Needs no database and no special hardware; installation takes a few commands.
- Uses free and open-source technologies only.
- Keeps analysis and presentation separate, so that either can be improved independently.
- Gives repeatable results because the random seeds are fixed.
- The node panel gives exact values for any user with a single click.

6.9 LIMITATIONS OF THE SYSTEM

The following limitations were identified by reading and running the code. They are stated openly so that the report describes the software as it is.

1. **Synthetic data only.** The application analyses generated users. There is no CSV upload or connection to real social-media data.
2. **No interaction or time analysis.** The features are user-level measures; there is no contact-to-contact interaction count and no trend over time.
3. **Fixed parameters.** The number of users (300), clusters (4) and neighbours (5) are constants in the code and cannot be changed from the interface. The value k = 4 is not validated by a measure such as the elbow method or the silhouette score.
4. **Re-Cluster repeats the same clustering.** Because the seeds are fixed, pressing the button returns the same numeric data and the same cluster labels; only the random user identifiers change.
5. **Legend is a labelling heuristic.** Each cluster is named after the first persona met in it; cluster 1 also contains four Lurkers and cluster 2 contains fifteen.
6. **Error recovery.** After a failed request the button stays disabled until the page is reloaded, and HTTP error status is not checked.
7. **Open CORS policy.** `allow_origins=["*"]` together with `allow_credentials=True` is suitable only for development.
8. **Unused dependencies.** `networkx` and `pydantic` are listed in `requirements.txt` but are not used directly by the code.
9. **Internet dependency for the interface.** The graph library, icons and font are loaded from CDNs.
10. **Desktop layout.** No responsive rules for small screens are defined in the style sheet.
11. **Naming difference.** The page title and sidebar use the name SocialSphere.

7. SYSTEM TESTING

Software testing is an important element of software quality assurance and represents the final review of specification, design and coding. The costs associated with a software failure are the motivating force for well-planned and thorough testing. Testing and debugging are different activities: testing finds defects, while debugging locates and removes them. This chapter describes the testing approach used for SocialCircle Analyzer, reports the tests that were actually executed and lists the manual test cases for the interface.

7.1 TYPES OF TESTING

Unit Testing

Unit testing checks the smallest parts of the program independently. For this project the units are `generate_mock_data()` and the individual steps of `get_network_data()`. Their outputs (row counts, value ranges, cluster labels, link counts) can be checked directly in Python.

Integration Testing

Integration testing checks that the units work together. The complete result of `get_network_data()` was checked for consistency between nodes and links, for example that every link refers to an existing node and that every node has exactly five outgoing links. Integration between the browser and the server (the fetch call and the drawing of the graph) is covered by the manual test cases.

Functional Testing

Functional testing compares the behaviour of the running application with the functional requirements of Chapter 2. It is performed through the browser interface following the test cases in Section 7.4.

Performance Testing

Performance testing measures the speed of the system. Only the analysis function was timed; see Section 7.5.

Acceptance Testing

User acceptance testing focuses on the fitness of the system for its intended use and is carried out by the users. It was not carried out as a formal exercise for this project.

7.2 TEST ENVIRONMENT

The backend tests were executed by loading the submitted `main.py` in Python 3.12.3 with pandas 3.0.2, NumPy 2.4.4 and scikit-learn 1.8.0 and calling its functions directly. In the test environment the FastAPI package was not installed, so the web layer (`FastAPI`, `CORSMiddleware`, `StaticFiles`, `FileResponse`) was replaced by empty stand-ins while the analysis code itself ran unchanged. The tests therefore prove the correctness of the analysis logic, but not the HTTP layer; the HTTP behaviour and the browser interface are covered by the manual test cases in Section 7.4.

7.3 BACKEND TEST CASES AND RESULTS

Table 7.1 lists the twenty-one backend test cases that were executed. All of them passed.

Table 7.1: Backend Test Cases and Observed Results

Test ID	Test case	Observed	Result
BT-01	generate_mock_data(300) returns 300 rows	rows=300	PASS
BT-02	DataFrame has the 9 expected columns	id, name, persona, posts_per_day, likes_given, followers, interest_tech, interest_sports, interest_fashion	PASS
BT-03	All numeric activity values are non-negative	min=0.000	PASS
BT-04	Interest flags contain only 0 or 1	values in {0,1}	PASS
BT-05	Only four personas are generated	Influencer, Lurker, Sports Fan, Techie	PASS
BT-06	User IDs are unique	unique=300	PASS
BT-07	Names follow User_1 ... User_300	User_1..User_300	PASS
BT-08	Same numeric data on repeated calls (fixed seed 42)	numeric columns identical	PASS
BT-09	User IDs differ on repeated calls (uuid4)	all 300 ids differ	PASS
BT-10	get_network_data returns keys nodes and links	nodes, links	PASS
BT-11	Response contains 300 nodes	nodes=300	PASS
BT-12	Response contains 1500 links (300 x 5 neighbours)	links=1500	PASS
BT-13	Cluster labels lie in {0,1,2,3}	0, 1, 2, 3	PASS
BT-14	Every link endpoint refers to an existing node id	all endpoints valid	PASS
BT-15	No node is linked to itself (include_self=False)	0 self-loops	PASS

Test ID	Test case	Observed	Result
BT-16	Every node has exactly five outgoing links	out-degree = 5 for all 300 nodes	PASS
BT-17	Node size val = 10 + followers/500	formula verified for all nodes	PASS
BT-18	Response is JSON serialisable	126750 characters	PASS
BT-19	Standardised features have mean ~0 and std ~1	mean=0, std=1 for each feature	PASS
BT-20	Cluster labels identical on repeated requests	labels identical	PASS
BT-21	Cluster sizes sum to 300	{0: 84, 1: 71, 2: 90, 3: 55}	PASS

7.4 MANUAL TEST CASES FOR THE USER INTERFACE

The interface tests below require a browser and were prepared from the code. The result column is left for the tester to complete when the application is run; no result has been entered in advance.

Table 7.2: Manual Test Cases for the Interface

Test ID	Action	Expected result	Result
UT-01	Open `http://localhost:8080`	Page loads; sidebar and graph appear	[ ]
UT-02	Observe the statistics after loading	Users = 300 and Connections = 1500	[ ]
UT-03	Observe the legend	Four entries: Influencer, Sports Fan, Techie, Lurker with four different colours	[ ]
UT-04	Compare node colours with the legend	Node colours correspond to the legend colours	[ ]
UT-05	Hover over a node	The user name (for example User_7) is shown as a label	[ ]
UT-06	Click a node	Detail panel appears with name, persona, followers, posts per day and likes; view zooms to the node	[ ]
UT-07	Click the × button of the panel	Panel is hidden	[ ]
UT-08	Click an empty area of the canvas	Panel is hidden	[ ]
UT-09	Click Re-Cluster	Button shows “Processing...” and is disabled, then returns; statistics remain 300 and 1500	[ ]
UT-10	Stop the server, then click Re-Cluster	Alert “Failed to load data. Is the backend running?” is shown	[ ]
UT-11	Resize the browser window	Graph canvas adjusts to the new size	[ ]
UT-12	Open `http://localhost:8080/api/network`	JSON with `nodes` and `links` is displayed	[ ]

Test ID	Action	Expected result	Result
UT-13	Open `http://localhost:8080/static/app.js`	JavaScript source is returned	[ ]

7.5 PERFORMANCE OBSERVATION

The function `get_network_data()` was run twenty times in the test environment (a single-CPU virtual machine). The mean execution time was **137.9 ms**, the fastest run 129.9 ms and the slowest 164.3 ms. The time includes generating the data, standardisation, K-Means, the neighbour graph and building the response, and excludes network transfer and browser drawing. The JSON response was 126,750 bytes. These figures are measurements of one environment and are given only to show that the analysis is fast enough for interactive use with 300 users; they say nothing about larger data sets.

7.6 REQUIREMENT COVERAGE

Table 7.3 relates each functional requirement of Chapter 2 to the test cases that check it. Backend tests were executed and passed; interface tests are prepared and await execution in a browser.

Table 7.3: Coverage of Functional Requirements by Test Cases

Requirement	Test cases	Status
FR-01 Generate 300 users	BT-01 to BT-07	Executed: passed
FR-02 Standardise features	BT-19	Executed: passed
FR-03 K-Means, four clusters	BT-13, BT-20, BT-21	Executed: passed
FR-04 Five nearest neighbours	BT-12, BT-15, BT-16	Executed: passed
FR-05 JSON through /api/network	BT-10, BT-11, BT-14, BT-17, BT-18; UT-12	Backend executed: passed; UT-12 pending
FR-06 Serve page and static files	UT-01, UT-13	Pending (needs browser and FastAPI)
FR-07 Force-directed graph	UT-04, UT-05, UT-11	Pending
FR-08 Statistics and legend	UT-02, UT-03	Pending
FR-09 Node detail panel	UT-06, UT-07, UT-08	Pending
FR-10 Re-Cluster button	UT-09; BT-08, BT-09, BT-20 (repeat behaviour)	Backend executed: passed; UT-09 pending

Requirement	Test cases	Status
FR-11 Failure message	UT-10	Pending

The requirements marked pending depend on the FastAPI web layer and the browser, which were not available in the test environment. Recording their results is the last step before the project is demonstrated.

7.7 OBSERVATIONS FROM TESTING

The following points were found during testing and inspection and are the basis of the limitations and future enhancements in this report.

- The numeric data and the cluster labels were identical for repeated requests (test BT-08 and BT-20), while the user identifiers changed every time (BT-09).
- Every node had exactly five outgoing links and no link joined a node to itself (BT-15, BT-16).
- Six users had their number of posts per day clipped to zero by `max(0, value)`; no other feature was clipped.
- The failure path of the interface leaves the Re-Cluster button disabled.

8. CONCLUSION

SocialCircle Analyzer was designed and implemented as a compact web application that turns a table of user-behaviour records into an interactive picture of similar users. On the server, a FastAPI application generates 300 synthetic users of four personas, standardises six features, groups the users into four clusters with the K-Means algorithm and links each user to the five most similar users. In the browser, a force-directed graph shows the clusters as coloured regions, the follower counts as node sizes and the details of any user in a side panel.

The results obtained by running the actual backend code show that the clustering separates the generated Influencers, Sports Fans and Techies cleanly and groups 55 of the 74 Lurkers into a cluster of their own, and that 98.7 % of the similarity links join users of the same cluster. The analysis of 300 users takes about 138 ms in the test environment, which is fast enough for interactive use.

The project met its objectives: a multi-feature data set was prepared and standardised, users were clustered with scikit-learn, the result was exposed through a REST endpoint, and it was presented as an interactive graph using only free, open-source technology. During the work the author gained practical experience in Python data analysis, in building a REST API, in the structure of the K-Means algorithm and in browser-side visualisation.

The report has also stated the limits of the prototype honestly. The data is synthetic, the parameters are fixed in the code, and there is no CSV upload, time analysis or contact-to-contact interaction analysis. These limits define the direction of the future work described in the next chapter. The clean structure of the code, in which data generation is a single function, makes it straightforward to replace the generator with real data in a later version.

9. FUTURE ENHANCEMENT

The following enhancements are **not implemented** in the present version. They are proposed because the current structure of the project makes them natural next steps.

Table 9.1: Proposed Future Enhancements

No.	Enhancement	Description
1	CSV / data upload	Add an upload control and a `POST` endpoint that reads a CSV file with pandas and validates its columns, replacing the synthetic generator with real interaction data.
2	Contact-level interaction analysis	Use records of messages or calls between people to compute interaction frequency, most active contacts and communication patterns.
3	Trend analysis over time	Store dated interactions and show how the interaction of each contact changes from month to month.
4	Choosing the number of clusters	Let the user select k and support the choice with the elbow method or the silhouette score.
5	Adjustable parameters	Expose the number of neighbours and users as controls in the interface.
6	Use of graph metrics	Use the already-listed `networkx` package to compute measures such as degree or community structure.
7	Filtering and search	Search for a user by name and filter the graph by cluster or persona.
8	Export of results	Download the cluster labels and statistics as CSV or the graph as an image.
9	Database storage	Store users and results in a database such as SQLite or MongoDB so that analyses can be saved and compared.
10	Better error handling	Check HTTP status codes, restore the button after a failure and show an on-page message instead of an alert.
11	Security	Restrict the CORS origins and add user login if the application is deployed publicly.
12	Responsive design	Add media queries so that the interface works on phones and tablets.
13	Offline use	Bundle the graph library, icons and font with the application so that an internet connection is not required.
14	Automated tests	Add unit tests for the backend functions and end-to-end tests for the interface.

10. BIBLIOGRAPHY

Reference Books

1. Wasserman, S. and Faust, K. (1994). *Social Network Analysis: Methods and Applications*. Cambridge University Press, Cambridge.
2. Newman, M. E. J. (2018). *Networks* (2nd ed.). Oxford University Press, Oxford.

Research Papers and Dissertation

1. Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8), 651–666.
2. Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
3. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, Vol. 1, 281–297.
4. Arthur, D. and Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, 1027–1035.
5. Cover, T. and Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
6. Fruchterman, T. M. J. and Reingold, E. M. (1991). Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11), 1129–1164.
7. Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
8. Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
9. McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, 56–61.
10. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures.* Doctoral dissertation, University of California, Irvine.

Websites (official documentation)

1. Python documentation – https://docs.python.org/3/
2. FastAPI documentation – https://fastapi.tiangolo.com
3. Uvicorn documentation – https://www.uvicorn.org

4. pandas documentation – https://pandas.pydata.org/docs/
5. NumPy documentation – https://numpy.org/doc/
6. scikit-learn user guide – https://scikit-learn.org/stable/user_guide.html
7. force-graph library – https://github.com/vasturiano/force-graph
8. MDN Web Docs (HTML, CSS, JavaScript, Fetch API) – https://developer.mozilla.org
9. Font Awesome – https://fontawesome.com
10. Google Fonts – https://fonts.google.com

11. APPENDIX

I. SCREEN SHOTS

No screen shots were supplied with the project files, and none have been fabricated. Each frame below is a placeholder marking where a screen shot of the running application should be inserted. Start the server as described in Section 6.4, capture the indicated screen and replace the placeholder frame.

[INSERT SCREENSHOT HERE]

Open http://localhost:8080 and capture the whole browser window.

Figure 11.1: Home Page of SocialCircle Analyzer (full window after loading)

[INSERT SCREENSHOT HERE]

Capture the sidebar showing Users = 300, Connections = 1500 and the four legend entries.

Figure 11.2: Sidebar with Statistics and Cluster Legend

[INSERT SCREENSHOT HERE]

Capture the graph after the layout has settled so that the four colour groups are visible.

Figure 11.3: Force-Directed Network Graph Coloured by Cluster

[INSERT SCREENSHOT HERE]

Click a large node (an Influencer) and capture the panel and the zoomed graph.

Figure 11.4: Node Detail Panel of a Selected User

[INSERT SCREENSHOT HERE]

Click a node from another cluster and capture the panel to show the different values.

Figure 11.5: Node Detail Panel of a Small-Follower User

[INSERT SCREENSHOT HERE]

Click Re-Cluster and capture the button showing the spinner and "Processing...".

Figure 11.6: Re-Cluster Button in the Processing State

[INSERT SCREENSHOT HERE]

Open http://localhost:8080/api/network in the browser and capture the JSON.

Figure 11.7: JSON Output of the /api/network Endpoint

[INSERT SCREENSHOT HERE]

Capture the terminal showing python main.py and the Uvicorn start-up message.

Figure 11.8: Server Started from the Command Prompt

II. SAMPLE CODING

backend/requirements.txt

```
fastapi
uvicorn
scikit-learn
pandas
numpy
networkx
pydantic
```

backend/main.py

```
# pyrefly: ignore [missing-import]
from fastapi import FastAPI
# pyrefly: ignore [missing-import]
from fastapi.middleware.cors import CORSMiddleware
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.neighbors import kneighbors_graph
from sklearn.preprocessing import StandardScaler
import random
import uuid
import os
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

def generate_mock_data(n_users=200):
```

```
    np.random.seed(42)
    random.seed(42)

    # Generate 4 distinct persona groups
    # 1. Active Techies
    # 2. Sports Fans (casual)
    # 3. Influencers (high followers, high posts)
    # 4. Lurkers (low posts, low followers, high likes)

    users = []

    for i in range(n_users):
        persona = random.choice(['Techie', 'Sports Fan', 'Influencer', 'Lurker'])

        if persona == 'Techie':
            posts = np.random.normal(5, 2)
            likes = np.random.normal(20, 5)
            followers = np.random.normal(300, 100)
            tech = 1
            sports = 0
            fashion = random.choice([0, 1])
        elif persona == 'Sports Fan':
            posts = np.random.normal(2, 1)
            likes = np.random.normal(10, 3)
            followers = np.random.normal(150, 50)
            tech = 0
            sports = 1
            fashion = random.choice([0, 1])
        elif persona == 'Influencer':
            posts = np.random.normal(10, 3)
            likes = np.random.normal(50, 15)
            followers = np.random.normal(5000, 1000)
            tech = random.choice([0, 1])
            sports = random.choice([0, 1])
            fashion = 1
        else: # Lurker
            posts = np.random.normal(0.5, 0.5)
            likes = np.random.normal(40, 10)
            followers = np.random.normal(50, 20)
            tech = random.choice([0, 1])
            sports = random.choice([0, 1])
            fashion = random.choice([0, 1])

        users.append({
            "id": str(uuid.uuid4())[:8],
            "name": f"User_{i+1}",
            "persona": persona,
            "posts_per_day": max(0, posts),
            "likes_given": max(0, likes),
            "followers": max(0, followers),
            "interest_tech": tech,
            "interest_sports": sports,
            "interest_fashion": fashion
        })

    return pd.DataFrame(users)

@app.get("/api/network")
def get_network_data():
    df = generate_mock_data(300)

    # Features for clustering
    features = ['posts_per_day', 'likes_given', 'followers', 'interest_tech',
'interest_sports', 'interest_fashion']
    X = df[features]

    # Standardize
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # K-Means Clustering
    n_clusters = 4
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    clusters = kmeans.fit_predict(X_scaled)
    df['cluster'] = clusters
```

```
    # Generate Graph using K-Nearest Neighbors
    # Create edges between similar nodes
    A = kneighbors_graph(X_scaled, n_neighbors=5, mode='connectivity', include_self=False)

    nodes = []
    for _, row in df.iterrows():
        nodes.append({
            "id": row['id'],
            "name": row['name'],
            "val": 10 + (row['followers'] / 500), # Node size based on followers
            "cluster": int(row['cluster']),
            "persona": row['persona'],
            "posts": float(row['posts_per_day']),
            "likes": float(row['likes_given']),
            "followers": float(row['followers'])
        })

    links = []
    # A is a sparse matrix
    rows, cols = A.nonzero()
    for u, v in zip(rows, cols):
        links.append({
            "source": df.iloc[u]['id'],
            "target": df.iloc[v]['id']
        })

    return {
        "nodes": nodes,
        "links": links
    }

# Mount static files
app.mount("/static", StaticFiles(directory="../static"), name="static")

@app.get("/")
def serve_index():
    return FileResponse("../static/index.html")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

static/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SocialSphere - AI Network Clustering</title>
    <!-- Google Fonts for modern typography -->
    <link href="https://fonts.googleapis.com/css2?
family=Outfit:wght@300;400;600;800&display=swap" rel="stylesheet">
    <!-- FontAwesome for icons -->
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    <link rel="stylesheet" href="/static/style.css">
    <!-- Force Graph Script -->
    <script src="https://unpkg.com/force-graph"></script>
</head>
<body>
    <div id="graph-container"></div>

    <div class="glass-panel sidebar">
        <div class="brand">
            <i class="fa-solid fa-network-wired"></i>
            <h1>SocialSphere</h1>
        </div>
        <p class="subtitle">K-Means Social Clustering</p>

        <div class="stats-container">
            <div class="stat-box">
                <span class="stat-value" id="stat-nodes">-</span>
                <span class="stat-label">Users</span>
```

```
            </div>
            <div class="stat-box">
                <span class="stat-value" id="stat-links">-</span>
                <span class="stat-label">Connections</span>
            </div>
        </div>

        <div class="cluster-legend">
            <h3>User Clusters</h3>
            <div id="legend-items">
                <!-- Legend items injected by JS -->
            </div>
        </div>

        <div class="controls">
            <button id="btn-recluster" class="primary-btn">
                <i class="fa-solid fa-rotate"></i> Re-Cluster Data
            </button>
        </div>
    </div>

    <!-- Node Detail Panel -->
    <div id="node-panel" class="glass-panel node-panel hidden">
        <button id="close-panel" class="close-btn"><i class="fa-solid
fa-xmark"></i></button>
        <div class="node-header">
            <div class="avatar" id="node-avatar">
                <i class="fa-solid fa-user"></i>
            </div>
            <div>
                <h2 id="node-name">User Name</h2>
                <span class="badge" id="node-persona">Persona</span>
            </div>
        </div>
        <div class="node-metrics">
            <div class="metric">
                <i class="fa-solid fa-users"></i>
                <div>
                    <span class="metric-val" id="node-followers">0</span>
                    <span class="metric-label">Followers</span>
                </div>
            </div>
            <div class="metric">
                <i class="fa-solid fa-pen"></i>
                <div>
                    <span class="metric-val" id="node-posts">0</span>
                    <span class="metric-label">Posts/Day</span>
                </div>
            </div>
            <div class="metric">
                <i class="fa-solid fa-heart"></i>
                <div>
                    <span class="metric-val" id="node-likes">0</span>
                    <span class="metric-label">Likes</span>
                </div>
            </div>
        </div>
    </div>

    <script src="/static/app.js"></script>
</body>
</html>
```

static/app.js

```
const colorPalette = {
    0: getComputedStyle(document.documentElement).getPropertyValue('--cluster-0').trim(),
    1: getComputedStyle(document.documentElement).getPropertyValue('--cluster-1').trim(),
    2: getComputedStyle(document.documentElement).getPropertyValue('--cluster-2').trim(),
    3: getComputedStyle(document.documentElement).getPropertyValue('--cluster-3').trim(),
};

// Initialize Force Graph
const elem = document.getElementById('graph-container');
const Graph = ForceGraph()(elem)
```

```
    .backgroundColor('#0b0f19')
    .nodeRelSize(4)
    .nodeVal('val')
    .nodeLabel('name')
    .nodeColor(node => colorPalette[node.cluster])
    .linkColor(() => 'rgba(255,255,255,0.1)')
    .linkWidth(0.5)
    .onNodeClick(node => {
        showNodePanel(node);
        // Center/zoom on node
        Graph.centerAt(node.x, node.y, 1000);
        Graph.zoom(4, 2000);
    })
    .onBackgroundClick(() => {
        hideNodePanel();
    });

// Fetch Data
async function loadNetworkData() {
    try {
        const btn = document.getElementById('btn-recluster');
        btn.innerHTML = '<i class="fa-solid fa-spinner fa-spin"></i> Processing...';
        btn.disabled = true;

        const response = await fetch('/api/network');
        const data = await response.json();

        Graph.graphData(data);

        // Update UI Stats
        document.getElementById('stat-nodes').textContent = data.nodes.length;
        document.getElementById('stat-links').textContent = data.links.length;

        // Create Legend
        createLegend(data.nodes);

        btn.innerHTML = '<i class="fa-solid fa-rotate"></i> Re-Cluster Data';
        btn.disabled = false;

        // Fit graph to view
        setTimeout(() => Graph.zoomToFit(400, 50), 500);

    } catch (error) {
        console.error("Failed to load network data:", error);
        alert("Failed to load data. Is the backend running?");
    }
}

function createLegend(nodes) {
    const legendContainer = document.getElementById('legend-items');
    legendContainer.innerHTML = '';

    // Extract unique personas and map them to their cluster
    const clusterMap = {};
    nodes.forEach(n => {
        if (!clusterMap[n.cluster]) {
            clusterMap[n.cluster] = n.persona;
        }
    });

    Object.keys(clusterMap).sort().forEach(cluster => {
        const item = document.createElement('div');
        item.className = 'legend-item';

        const color = colorPalette[cluster];

        item.innerHTML = `
            <div class="legend-color" style="background: ${color}; color: ${color}"></div>
            <span>${clusterMap[cluster]}</span>
        `;
        legendContainer.appendChild(item);
    });
}

// UI Interactions
```

```
function showNodePanel(node) {
    const panel = document.getElementById('node-panel');
    panel.classList.remove('hidden');

    document.getElementById('node-name').textContent = node.name;
    document.getElementById('node-persona').textContent = node.persona;
    document.getElementById('node-followers').textContent =
Math.round(node.followers).toLocaleString();
    document.getElementById('node-posts').textContent = node.posts.toFixed(1);
    document.getElementById('node-likes').textContent =
Math.round(node.likes).toLocaleString();

    const avatar = document.getElementById('node-avatar');
    avatar.style.background = colorPalette[node.cluster];
}

function hideNodePanel() {
    document.getElementById('node-panel').classList.add('hidden');
}

document.getElementById('close-panel').addEventListener('click', hideNodePanel);
document.getElementById('btn-recluster').addEventListener('click', loadNetworkData);

// Initial Load
loadNetworkData();

// Handle window resize
window.addEventListener('resize', () => {
    Graph.width(window.innerWidth);
    Graph.height(window.innerHeight);
});
```

static/style.css

```
:root {
    --bg-color: #0b0f19;
    --panel-bg: rgba(17, 24, 39, 0.7);
    --panel-border: rgba(255, 255, 255, 0.08);
    --text-main: #f3f4f6;
    --text-muted: #9ca3af;
    --accent: #6366f1;
    --accent-hover: #4f46e5;

    /* Cluster Colors */
    --cluster-0: #f43f5e; /* Rose */
    --cluster-1: #3b82f6; /* Blue */
    --cluster-2: #10b981; /* Emerald */
    --cluster-3: #8b5cf6; /* Violet */
}

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Outfit', sans-serif;
}

body {
    background-color: var(--bg-color);
    color: var(--text-main);
    overflow: hidden;
    width: 100vw;
    height: 100vh;
}

#graph-container {
    position: absolute;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    z-index: 1;
}

/* Glassmorphism utility */
```

```
.glass-panel {
    background: var(--panel-bg);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid var(--panel-border);
    border-radius: 16px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.4);
    z-index: 10;
}

/* Sidebar */
.sidebar {
    position: absolute;
    top: 24px;
    left: 24px;
    width: 320px;
    padding: 24px;
    display: flex;
    flex-direction: column;
    gap: 24px;
}

.brand {
    display: flex;
    align-items: center;
    gap: 12px;
}

.brand i {
    font-size: 28px;
    color: var(--accent);
}

.brand h1 {
    font-size: 24px;
    font-weight: 800;
    letter-spacing: -0.5px;
    background: linear-gradient(135deg, #fff, #a5b4fc);
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
}

.subtitle {
    color: var(--text-muted);
    font-size: 14px;
    margin-top: -16px;
}

/* Stats */
.stats-container {
    display: flex;
    gap: 12px;
}

.stat-box {
    flex: 1;
    background: rgba(255, 255, 255, 0.03);
    border: 1px solid rgba(255, 255, 255, 0.05);
    border-radius: 12px;
    padding: 16px;
    display: flex;
    flex-direction: column;
    align-items: center;
}

.stat-value {
    font-size: 24px;
    font-weight: 800;
    color: #fff;
}

.stat-label {
    font-size: 12px;
    color: var(--text-muted);
```

```
    text-transform: uppercase;
    letter-spacing: 1px;
    margin-top: 4px;
}

/* Legend */
.cluster-legend h3 {
    font-size: 14px;
    text-transform: uppercase;
    letter-spacing: 1px;
    color: var(--text-muted);
    margin-bottom: 12px;
}

.legend-item {
    display: flex;
    align-items: center;
    gap: 12px;
    margin-bottom: 10px;
    font-size: 14px;
}

.legend-color {
    width: 14px;
    height: 14px;
    border-radius: 50%;
    box-shadow: 0 0 10px currentColor;
}

/* Buttons */
.primary-btn {
    width: 100%;
    padding: 14px;
    border: none;
    border-radius: 10px;
    background: var(--accent);
    color: white;
    font-size: 15px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.2s ease;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
}

.primary-btn:hover {
    background: var(--accent-hover);
    transform: translateY(-2px);
    box-shadow: 0 8px 16px rgba(99, 102, 241, 0.3);
}

.primary-btn:active {
    transform: translateY(0);
}

/* Node Detail Panel */
.node-panel {
    position: absolute;
    top: 24px;
    right: 24px;
    width: 300px;
    padding: 24px;
    transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
    transform: translateX(0);
    opacity: 1;
}

.node-panel.hidden {
    transform: translateX(120%);
    opacity: 0;
    pointer-events: none;
}
```

```
.close-btn {
    position: absolute;
    top: 16px;
    right: 16px;
    background: transparent;
    border: none;
    color: var(--text-muted);
    font-size: 18px;
    cursor: pointer;
    transition: color 0.2s;
}

.close-btn:hover {
    color: white;
}

.node-header {
    display: flex;
    align-items: center;
    gap: 16px;
    margin-bottom: 24px;
}

.avatar {
    width: 50px;
    height: 50px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 20px;
    color: white;
}

.node-header h2 {
    font-size: 18px;
    margin-bottom: 4px;
}

.badge {
    font-size: 11px;
    padding: 3px 8px;
    border-radius: 20px;
    background: rgba(255, 255, 255, 0.1);
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.node-metrics {
    display: flex;
    flex-direction: column;
    gap: 16px;
}

.metric {
    display: flex;
    align-items: center;
    gap: 16px;
    background: rgba(255, 255, 255, 0.03);
    padding: 12px;
    border-radius: 10px;
}

.metric i {
    font-size: 20px;
    color: var(--accent);
    width: 24px;
    text-align: center;
}

.metric-val {
    display: block;
    font-size: 18px;
    font-weight: 600;
}
```

```
.metric-label {
    font-size: 12px;
    color: var(--text-muted);
}
```

III. SAMPLE API OUTPUT

The listing below shows the beginning of the real response of `get_network_data()` when it was executed in the test environment (Section 6.7). The two first nodes and the first three links are shown; the complete response contains 300 nodes and 1,500 links. The `id` values are random and differ at every request.

```
// abridged and rounded to 4 decimals: first 2 of 300 nodes, first 3 of 1500 links
{
  "nodes": [
    {
      "id": "8f9d4419",
      "name": "User_1",
      "val": 10.7295,
      "cluster": 2,
      "persona": "Techie",
      "posts": 5.9934,
      "likes": 19.3087,
      "followers": 364.7689
    },
    {
      "id": "9c876042",
      "name": "User_2",
      "val": 19.5317,
      "cluster": 0,
      "persona": "Influencer",
      "posts": 14.5691,
      "likes": 46.4877,
      "followers": 4765.863
    }
  ],
  "links": [
    {
      "source": "8f9d4419",
      "target": "cbc7dfe1"
    },
    {
      "source": "8f9d4419",
      "target": "0999b826"
    },
    {
      "source": "8f9d4419",
      "target": "e932af06"
    }
  ]
}
```

IV. SAMPLE OF THE GENERATED DATA

Table 11.1 lists the first ten users of the test run together with the cluster assigned by K-Means. The identifiers are omitted because they change at every request.

Table 11.1: First Ten Users of the Test Run

Name	Persona	Cluster	Posts / day	Likes	Followers
User_1	Techie	2	5.99	19.31	364.77

Name	Persona	Cluster	Posts / day	Likes	Followers
User_2	Influencer	0	14.57	46.49	4,765.86
User_3	Sports Fan	1	3.58	12.30	126.53
User_4	Techie	2	6.09	17.68	253.43
User_5	Techie	2	5.48	10.43	127.51
User_6	Techie	2	3.88	14.94	331.42
User_7	Sports Fan	1	1.09	5.76	223.28
User_8	Sports Fan	1	1.77	10.20	78.76
User_9	Sports Fan	1	1.46	10.33	92.45
User_10	Influencer	0	11.13	40.99	4,708.31

V. GLOSSARY OF TERMS

Table 11.2: Glossary of Technical Terms

Term	Meaning
API	Application Programming Interface; a defined way for one program to request services from another.
ASGI	Asynchronous Server Gateway Interface; the standard through which Python web applications and servers communicate.
Centroid / cluster centre	The mean position of all points in a cluster.
Cluster	A group of records that are more similar to each other than to records in other groups.
CORS	Cross-Origin Resource Sharing; a browser rule about which web addresses may call a server.
DataFrame	The pandas table structure with labelled rows and columns.
Euclidean distance	The straight-line distance between two points, calculated from the differences in every feature.
Feature	A measurable property of a record used in the analysis, such as followers.
Force-directed layout	A graph-drawing method in which nodes repel each other and links act as springs.
JSON	JavaScript Object Notation; a text format for structured data.
K-Means	A clustering algorithm that repeatedly assigns points to the nearest centre and recomputes the centres.
k-nearest neighbours	The k records that are closest to a given record.
Node and link	A point and a connecting line in a graph; here a user and a similarity connection.

Term	Meaning
Persona	One of the four behaviour types (Techie, Sports Fan, Influencer, Lurker) used to generate the synthetic users.
REST	A style of web interface in which resources are requested through URLs with HTTP methods.
Standardisation	Rescaling a feature to mean 0 and standard deviation 1.
Synthetic data	Data created by a program instead of being collected from real users.
Virtual environment	An isolated folder that holds the Python packages of one project.